

Cost and Precision of Spotlight SAR Image Formation on Google TPUs, an Nvidia GPU and a CPU

Paul Singerman
Penn State University
pgs16@psu.edu

Tom Braum
Penn State University
trb5864@psu.edu

October 2026

Abstract

A TPU is cheap only on dense bfloat16 matrix products, and SAR backprojection is a gather. Whether spotlight image formation can be written in the TPU’s idiom, and what it then costs against a GPU and a CPU, was measured on three Umbra collections at native size, formed on a TPU v5e, a TPU v6e, an Nvidia L4 and a 16-vCPU AMD instance by exact backprojection, factorized backprojection and polar format, with every image scored against a float64 exact backprojection. The factorized algorithm ran first as one JAX program on every device and then with kernels written for each accelerator from a stage profile (Pallas on the TPUs, CUDA on the L4), which made it 1.8 times faster on the TPUs and 3.8 times faster on the L4 at float32 accuracy without changing the images. With those kernels a thousand Panama images at float32 accuracy cost \$0.76 on the L4, \$1.79 on the v5e and \$2.09 on the v6e, and 54 times the L4’s figure on the CPU. The v6e is the fastest device, 1.4 times faster than the L4 per image at 3.8 times the hourly price. The L4’s float16 tensor-core final stage changes the image by 0.2 dB and the cost by 12%; the TPUs’ native single-pass products add 12 dB of error and leave phase errors beside bright returns that a coherent product would show. Corrected polar format costs \$0.42 per thousand on the L4, 1.8 times less than factorized backprojection there, but its correction runs 51 times slower on a TPU and the corrected image carries a -32 dB residual whose source was not found.

1 Introduction

A spotlight SAR processor now has a choice of hardware it did not have five years ago. The cloud rents GPUs and TPUs by the hour at prices set by the machine-learning market, and a phase history of 2×10^8 complex samples forming 10^8 pixels is, on paper, the regular arithmetic those devices were built for.

The difficulty is that they were built for different arithmetic. A GPU is fast on the operation backprojection performs, an independent per-pixel loop that reads a range profile at a computed address. A TPU does that operation on its vector unit at a rate two orders of magnitude lower, and is fast only when the work is a dense matrix product with bfloat16 operands. The question for a shop deciding where to put its quota is therefore not which device has the higher peak rate. It is whether image formation can be rewritten in the TPU’s idiom without losing accuracy, and what the rewritten algorithm then costs against a GPU running the natural one.

That question was put to three Umbra collections at the vendor’s native size, formed on four instances by three algorithms and scored against a float64 exact backprojection. The answer depends on how far each device’s code is taken. As one JAX program, the factorized algorithm cost about

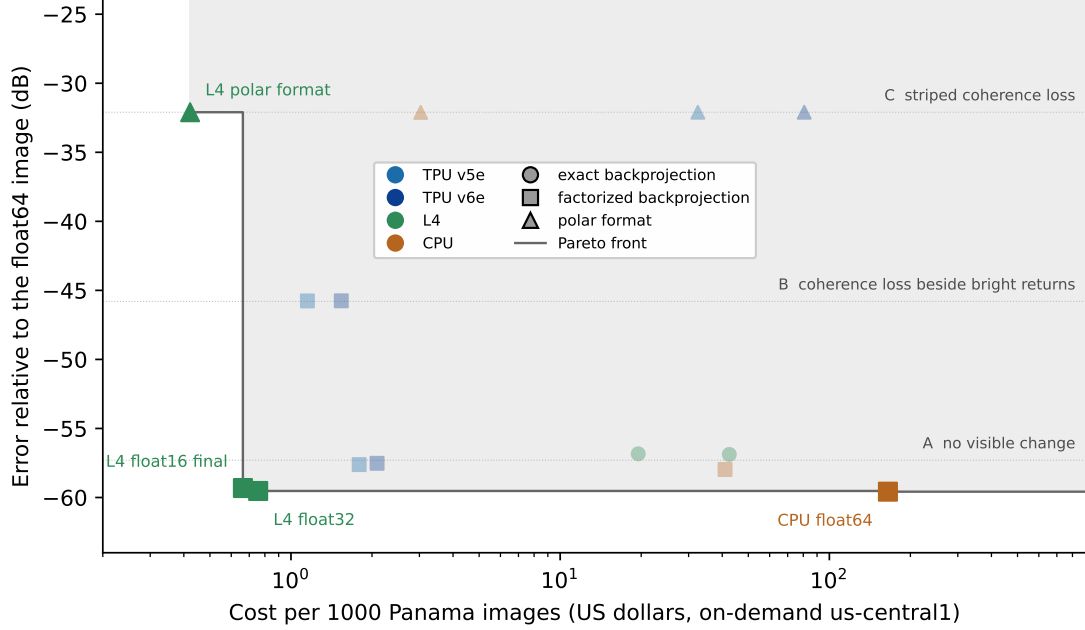


Figure 1: Cost per thousand Panama Canal images against the error of the image relative to the float64 reference, for the selected configuration of each device, algorithm and precision (Table 3). Cost and error are defined in Section 5. The line is the Pareto front, the 4 configurations that no other configuration beats on both cost and error; only these are labeled, and the dominated configurations are drawn faded. Polar format is shown with its geometric correction; without it the image is displaced by up to 18 pixels (Section 6.3). The dotted lines mark the three groups of Table 3 by the change a viewer sees against the reference.

the same on the two TPUs and the L4 at float32 accuracy, within 11% on Panama. Kernels written for each device from a stage profile (Pallas on the TPUs, CUDA on the L4; Appendix D) sped the TPUs up by 1.8 times and the L4 by 3.8 times at that accuracy. The L4 then forms the float32-accuracy image for \$0.76 per thousand against \$1.79 and \$2.09 on the v5e and v6e, and the CPU costs 54 times the L4 (Table 3). The v6e remains the fastest device. What separates the accelerators beyond that is how each behaves when the arithmetic is lowered (Figure 1), and polar format, once its distortion is corrected, is the cheapest algorithm on the GPU and the slowest on a TPU (Section 6.3). An earlier internal round on smaller, simulated data is superseded here and is referred to below as the first round.

Figures 2 to 4 show the images: the three scenes, four Panama regions across devices and algorithms with their coherence against the reference, and three regions at pixel scale (0.41 by 0.36 m). In amplitude no image differs visibly from the reference; the differences are in the complex values, where the coherence maps show them.



Figure 2: The three collections as formed by the float64 reference, downsampled for display (amplitude in decibels over a 45 dB range, azimuth horizontal, range vertical, 1 km bar). The yellow boxes on the Panama Canal image are the four 512 by 512 pixel regions of Figure 3.

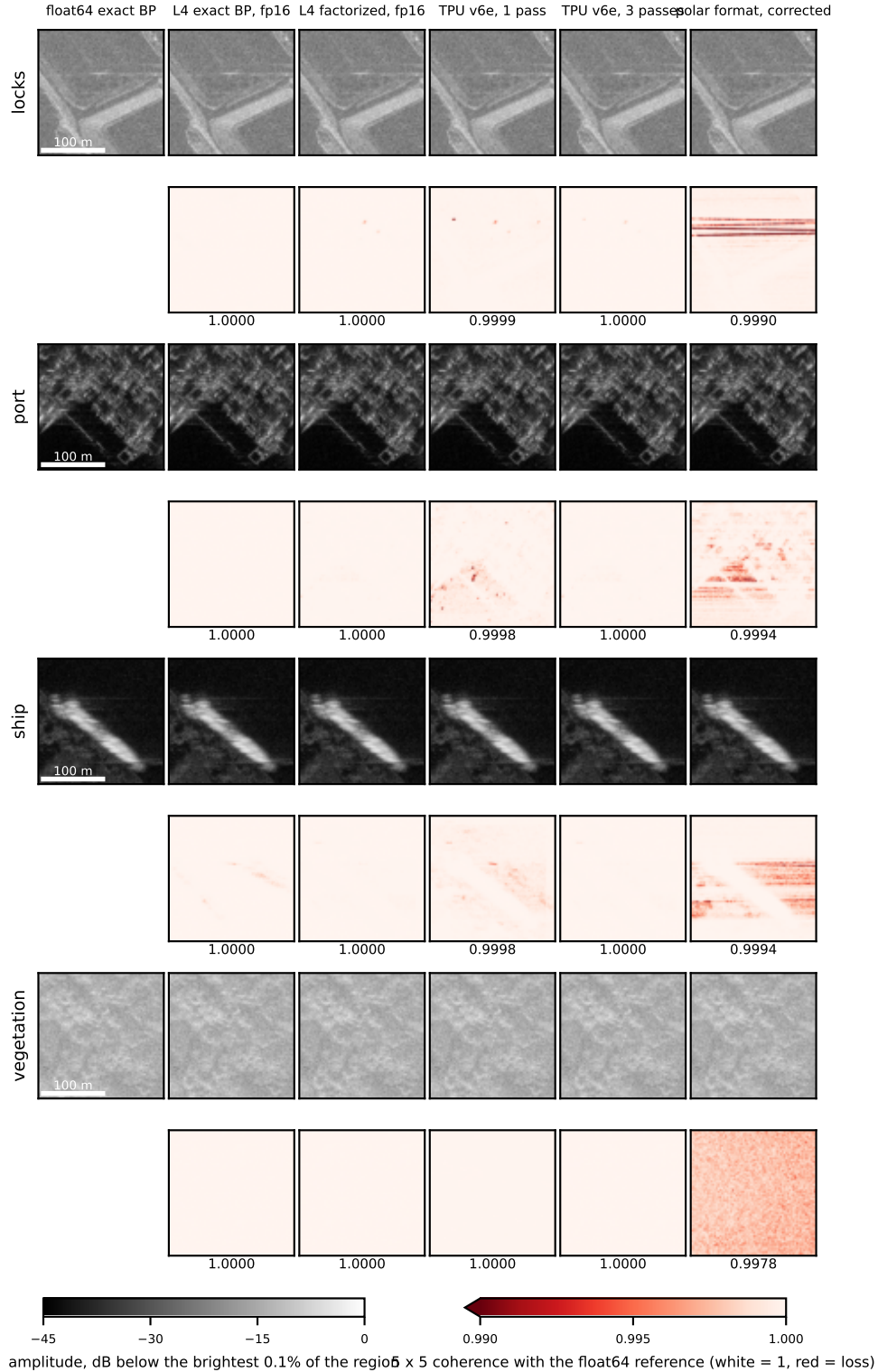


Figure 3: Panama Canal, four regions of 512 by 512 pixels at full resolution (100 m bar). Top of each pair: amplitude over a 45 dB range for the float64 reference, the L4 exact backprojection (float16 profiles), the L4 factorized image with float16 products, the TPU v6e factorized image with single-pass and three-pass products, and polar format with its geometric correction. Bottom: the 5 by 5 coherence of each image with the reference from 0.99 (red) to 1 (white), with its mean below each panel. The regions are the Cocolí Locks, the Port of Balboa, a ship in the channel and vegetated terrain 2.7 km from the scene center; Appendix G locates them.

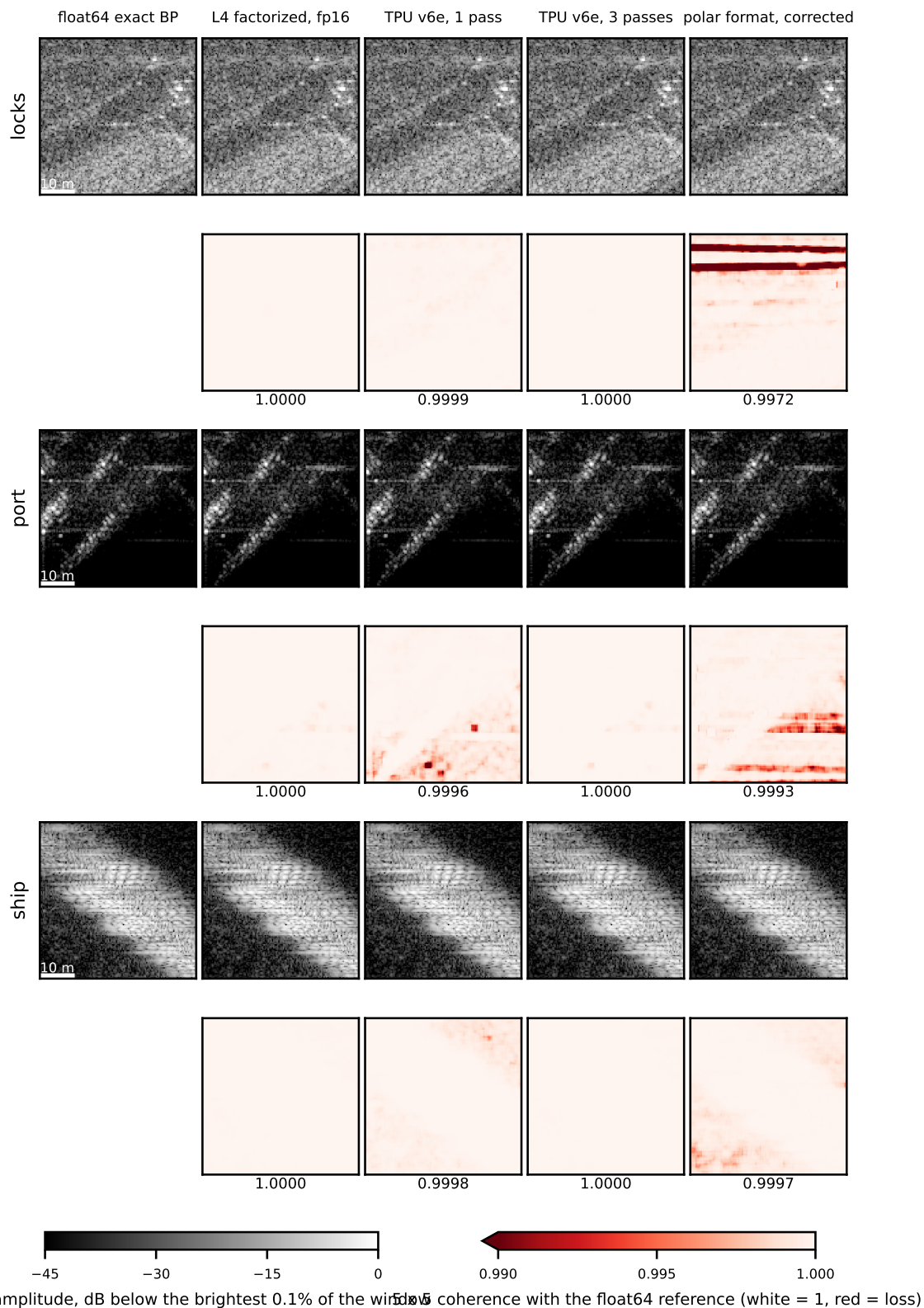


Figure 4: Panama Canal at pixel scale: 128 by 128 pixel windows (53 m in azimuth by 46 m in slant range, 10 m bar) around the brightest feature of the lock, port and ship regions of Figure 3, drawn without interpolation so that each pixel (0.41 by 0.36 m) is visible on the page. Columns, scales and coherence maps as in Figure 3, without the L4 exact backprojection, which does not differ visibly from the reference at this scale either.

2 Prior work

Backprojection on parallel hardware has a fifteen-year literature, almost all of it on GPUs and almost none of it on cost. Fasih and Hartley gave the first single-GPU CUDA implementation [4]; Benson, Campbell and Cook scaled it to gigapixel images on a GPU cluster and reported throughput as a function of precision [5]; Park and colleagues reached 35 billion backprojections per second per node on Xeon and Xeon Phi processors by strength reduction and locality optimization [6]; Cordes and Leiser ran the kernel on FPGAs with general-purpose hosts [7]. On the accuracy side, Capozzoli, Curcio and Lisenò replaced the linear interpolation of the range profiles with a NUFFT and showed it more accurate at similar cost on a GPU [8], and Xu and colleagues recently ran a streaming GPU backprojection in float16 [9]. These papers establish that the GPU is the natural home of the exact algorithm; none measures what an image costs in dollars, and none compares against a device that cannot gather.

The two approximate algorithms used here are older. Fast factorized backprojection descends from McCorkle’s quadtree [21] and Ulander’s formulation [23], was applied to strongly nonlinear airborne tracks by Frey and colleagues [10], and was put on GPUs for circular SAR by Ponce and colleagues [11]. Polar format is the textbook algorithm [24, 25]; its scene-size limit from wavefront curvature was derived by Doerry together with a post-processing compensation [12], and derived again in terms of the residual quadratic phase error by Gorham and Rigling [13]. Jakowatz and Doren compared polar format with backprojection for spotlight images in 2006 [14]. The corrected polar format of this report follows that line: form the image fast, then remove the known distortion.

The TPU has been used outside machine learning only where a problem can be written as dense matrix products. Jouppi and colleagues describe the architecture and its matrix units [15], and the fourth generation’s sparse cores for embedding lookups [16]. Lu and colleagues formulated large discrete Fourier transforms as Kronecker products of dense matrices on TPU clusters [17] and then the nonuniform FFT, reporting 20 to 80 times a CPU on one TPU card [18]; Lewis and colleagues ran distributed dense linear algebra at supercomputer scale on TPU pods [19]. Those results are the reason the factorized algorithm here is written entirely as matrix products. What the literature does not contain is a measurement of SAR image formation on a TPU at all, or a cost comparison between TPU, GPU and CPU on real spotlight collections at native size against a common float64 reference. That is this report’s contribution.

3 Data

Three collections from Umbra’s open data catalog [1] were used (Table 1). They are the Pacific entrance of the Panama Canal (2023-07-18, 02:30 UTC), with the Cocolí Locks, the navigation channel and the Port of Balboa; central Melbourne (2023-02-08); and farmland near Johnston, Iowa (2023-10-20). Each was processed from the compensated phase history (CPHD) as delivered, apart from 43 leading pulses of the Melbourne collection that carry no positions or samples. The image grid is the vendor’s own: the pixel counts, spacings and slant image plane of the vendor’s complex image (SICD).

The four Panama regions of Figure 3 were identified against Sentinel-2 imagery and six other Umbra passes, which also show that the bright target of the ship region is a ship moving at 3.0 m/s, imaged 194 m from its position (Appendix G).

Table 1: The three collections and their native image grids. Spacings are azimuth by range in the slant plane; squint is the Doppler cone angle less 90° at the aperture center; range is the slant range to the scene center at the start of the aperture.

Scene	Image (az $\times$ rg)	Spacing (m)	Pulses $\times$ samples	Bandwidth (MHz)	Graze	Squint	Range (km)
Panama Canal	$12,207 \times 8,808$	0.413×0.356	$15,186 \times 14,399$	337	42.6°	-28.1°	738
Melbourne	$10,077 \times 10,273$	0.422×0.298	$15,948 \times 14,399$	405	43.4°	0.6°	760
Iowa farmland	$8,521 \times 8,465$	0.517×0.328	$13,003 \times 11,663$	356	50.3°	2.8°	653

Table 2: Instances, the arithmetic timed on each (Section 4 and Appendix B) and on-demand prices (US dollars per hour, us-central1).

Label	Instance	Arithmetic timed	Price
TPU v5e	1 chip (v5litepod-1), 24 vCPU host	bfloat16 single-, three- and six-pass products; float32 or bfloat16 data	1.200
TPU v6e	1 chip (v6e-1), 44 vCPU host	bfloat16 single-, three- and six-pass products; float32 or bfloat16 data	2.700
L4	g2-standard-4, 1 Nvidia L4 (24 GB)	float16, TF32 and float32 products; float16 or float32 data	0.707
CPU	c4d-highmem-16, AMD EPYC 9B45, 8 cores, 126 GB	float32 and float64	0.964

4 Devices, algorithms and arithmetic

Table 2 lists the instances and their on-demand us-central1 list prices (Google Cloud Billing Catalog, 1 October 2026). The CPU instance is the smallest that holds the float64 factorized image; the L4 instance is the smallest offered with that GPU. Software was JAX 0.11.2 throughout, with CUDA 12 and CuPy 14.2.0 on the L4 and Numba 0.68.0 for the reference.

Three algorithms were run (Appendix A). *Exact backprojection* sums, for every pixel, the range-compressed pulse interpolated at the pixel’s range [20]: one indexed read per pixel and pulse, 1.6 trillion for the Panama image. A CUDA kernel runs it on the L4 with 8 times oversampled profiles in float32 or float16; a float64 CPU version with 16 times oversampling is the reference. On the TPU the same gather runs at 4×10^7 to 10^8 reads per second depending on the access pattern, against about 10^{10} on the L4 (Appendix C), so exact backprojection was not run there.

Factorized backprojection divides the image into tiles, re-centers the phase history on each tile with a phase ramp, filters and decimates it, and recurses [21, 23]. Every stage is an element-wise or a dense matrix product, the operation the TPU’s matrix units are built for. One JAX program runs on all four devices; it is the CPU form and the baseline on the accelerators. Where it is faster, strided convolutions replace the decimation matrices (*conv. filters* in the tables), and the phase ramps are evaluated directly or from a coarse and a fine table (*direct* and *split-table* ramps).

The accelerator rows of Table 3 use kernels written for each device from a stage profile of that program, Pallas on the TPUs and CUDA on the L4 (*fused kernels* and *CUDA kernels* in the tables). The rotated phase history of each child and the ramp matrices of the final stage stay on chip instead of passing through memory, and the final stage’s ramps come from a complex recurrence instead of one sine per sample. Appendix D gives the diagnosis, the kernels and the time each step removed. The images are unchanged to within the rounding of the precision concerned. The tiles’ band limits leave a residual against exact backprojection that is independent of the arithmetic (Table 4, float64 row).

Polar format resamples the pulses and samples onto a rectangular wavenumber grid by chirp-z transforms [26] and forms the image by one FFT [24, 25]; it is the cheapest of the three (Table 3). It assumes a planar wavefront, so scatterers are displaced by an amount that grows with the square of their distance from the scene center; a geometric correction resamples the image through the fitted displacement map (Appendix E). Its pulse resampling runs as a dense product or as a sum

over gathered rows (*gather*).

The TPU’s matrix units multiply bfloat16 operands only. A float32 product is formed from one bfloat16 pass by default, which gives bfloat16 accuracy, or from three or six passes (JAX precision *default*, *high*, *highest*), called single-, three- and six-pass products below. The L4’s tensor cores take float16 operands (11 significant bits against bfloat16’s 8), so *sixteen-bit* means a more accurate type on the L4 than on the TPU. The L4’s default float32 product uses TF32 operands (also 11 bits); the configurations called *float32* on the L4 use full float32. Every configuration keeps the geometry in float32 or float64 and lowers only the storage type of the data and the operand type of the products; the CPU runs native float32 and float64 (Appendix B).

5 Measurement protocol

Device time per image is the wall time of one image on a warm device with the data resident, compilation excluded and the per-collection host work included (about 0.5 s). Throughput is the number of images per hour of a pipelined loop in which the transfers of the next collection and the previous image overlap the formation of the current one. Cost per image is the instance price divided by the throughput; where no pipelined loop was run, the inverse device time is used and the entry carries an asterisk. Repeated timings agree within 4%, and the L4’s pipelined loops run 6 to 11% slower per image than its device time (Appendix F).

Every image is compared with the float64 exact backprojection of the same collection after one gain has been fitted per algorithm family (complex for the backprojections, real for polar format). The error quoted is the energy of the difference relative to the energy of the reference (dB). The largest pixel difference, the 5 by 5 coherence and the amplitude and phase deviations over the brighter half of the pixels are tabulated with it. The selected configuration of each device, algorithm and precision is the fastest implementation form whose image is within 0.1 dB of the other forms on that device. Power was measured on the L4 only, from `nvidia-smi`.

6 Results

6.1 Time and cost per image

A single error figure in decibels does not tell an analyst whether an image is usable, because a 10 dB change in the residual can be invisible in amplitude and obvious in a coherence product. Table 3 therefore sorts the configurations by what a viewer would notice against the float64 image, with thresholds taken from the statistics of Table 4. Group A shows no visible change: the 99th-percentile amplitude deviation is within 3 dB and the 5 by 5 coherence stays above 0.999 even at its 0.1 percentile. Group B has the same amplitude image but loses coherence below 0.999 at the darkest tenth of a percent of pixels, which Figures 3 and 4 show as a halo beside bright returns. Group C loses coherence below 0.99 in stripes.

Polar format without its geometric correction is omitted from the table: an image whose features sit up to 18 pixels from their true position is not an operational product, however fast it was formed (Section 6.3).

The spread inside group A is large, \$0.66 to \$165 per thousand, and almost all of it is exact backprojection and the CPU. Group B, the TPUs’ native arithmetic, opens at \$1.15 on the v5e, and group C at \$0.42 with corrected polar format on the L4. The last column marks the 4 configurations on the Pareto front of Figure 1, the ones nothing else beats on both cost and error.

Table 3: Panama Canal: the selected configurations grouped by the change a viewer sees against the float64 image, cheapest first within each group. Cost and error as defined in Section 5; an asterisk marks rows timed without a pipelined loop, costed from the device time. A bullet marks the Pareto-optimal rows of Figure 1: no other configuration is both cheaper and closer to the reference. The groups are defined in Section 6; throughput and the other two collections are in Table 10.

Device	Algorithm	Arithmetic	Time (s)	\$ per 1000	Error (dB)	Pareto
<i>A. No visible change: amplitude and coherence maps match the reference</i>						
L4	Factorized BP	float32 levels, float16 final, CUDA kernels	3.2	0.66	−59.3	•
L4	Factorized BP	float32, CUDA kernels	3.6	0.76	−59.5	•
TPU v5e	Factorized BP	three-pass products, fused kernels	5.1	1.79	−57.6	
TPU v6e	Factorized BP	three-pass products, fused kernels	2.6	2.09	−57.5	
L4	Exact BP	float16 profiles	103.8	19.50	−56.8	
CPU	Factorized BP	float32, conv. filters, direct ramps	217.4	40.97	−58.0	
L4	Exact BP	float32 profiles	216.4	42.49	−56.9	
CPU	Factorized BP	float64, conv. filters, direct ramps	616.3	165.03*	−59.6	•
<i>B. Amplitude unchanged; coherence loss beside bright returns</i>						
TPU v5e	Factorized BP	single-pass products, fused kernels	3.2	1.15	−45.8	
TPU v6e	Factorized BP	single-pass products, fused kernels	1.9	1.54	−45.8	
<i>C. Amplitude unchanged; striped coherence loss</i>						
L4	Polar format	float32, gather, corrected	2.1	0.42	−32.1	•
CPU	Polar format	float32, gather, corrected	11.3	3.03*	−32.1	
TPU v5e	Polar format	float32, gather, corrected	97.4	32.48*	−32.1	
TPU v6e	Polar format	float32, corrected	105.3	80.66	−32.1	

The factorized rows are the kernel builds of Appendix D, at the error of the JAX program’s images. That program took 4.6, 9.3 and 13.7 s for the float32-accuracy Panama image on the v6e, v5e and L4 (Table 8) and cost within 11% across the three devices. The kernels made both TPUs 1.8 times faster at that accuracy and the L4 3.8 times, so the comparison that follows is between devices each running the fastest form found for it.

The v6e forms the factorized image fastest, in 1.9 s with single-pass products and 2.6 s with three-pass products (Table 3; other collections in Table 10, every timed configuration in Appendix H). The v5e takes 3.2 and 5.1 s for the same two, and the L4 3.6 s in float32 and 3.2 s with the float16 final stage. The hourly price rises faster than the device speed: the v6e is 3.8 times the L4’s price for 1.4 times its speed.

At float32 accuracy a thousand Panama images therefore cost \$0.76 on the L4, \$1.79 on the v5e and \$2.09 on the v6e. The order holds on every collection (Table 10): on Melbourne the three figures (L4, v5e, v6e) are \$0.78, \$1.82 and \$1.93, on Iowa \$0.50, \$1.22 and \$1.51. The CPU, at \$40.97 with four pinned processes (Section 7), costs 54 times the L4; its program is the JAX one, so a CPU kernel round would narrow that ratio by some factor below the accelerators’ two to four.

With the sixteen-bit products each device offers, the L4 costs \$0.66, the v5e \$1.15 and the v6e \$1.54, for images that differ by 13.6 dB in error (Section 6.2). On the L4 only the final stage runs in float16, on the tensor cores, and the image stays in group A; on the TPUs every product does. Exact backprojection on the L4 costs \$19.50 per thousand with float16 profiles and \$42.49 with float32, 29 and 56 times the cost of the factorized image of the same precision class on the same card.

Table 4: Precision of the Panama Canal images against the float64 exact backprojection: energy of the difference (dB), largest pixel difference relative to the brightest pixel (dB), minimum 5 by 5 coherence (the mean and 0.1 percentile are in Appendix H), amplitude ratio (99th percentile and maximum, dB) and phase difference (99th percentile and maximum, degrees) over the brighter half of the pixels. The two 16 times oversampled rows share the reference interpolation and read better than their estimated true error of about -69 dB (Section 6.2). The L4 float32 row is the dense-filter image; the convolution form applies the same filter coefficients. The ramp form does not alter the image; TPU rows are labeled with the form whose image was saved.

Device	Configuration	Error	Max	Coh. min	Amp. p99	max	Phase p99	max
L4	Exact BP, float32 profiles	-56.9	-62.2	0.923	0.03	2.5	0.2	15
L4	Exact BP, float16 profiles	-56.8	-62.2	0.922	0.03	2.5	0.2	15
L4	Exact BP, float32 profiles, 16x oversampled	-78.5	-85.5	1.000	0.00	0.1	0.0	0
L4	Exact BP, float16 profiles, 16x oversampled	-74.5	-85.2	1.000	0.01	0.1	0.0	1
L4	Factorized BP, float32, CUDA kernels	-59.5	-64.7	0.886	0.03	3.2	0.2	18
L4	Factorized BP, float32 levels, float16 final, CUDA kernels	-59.3	-64.4	0.886	0.03	3.2	0.2	18
L4	Factorized BP, float32	-57.3	-64.3	0.886	0.03	3.4	0.2	18
L4	Factorized BP, TF32 products	-56.8	-65.0	0.842	0.04	3.2	0.2	20
L4	Factorized BP, float16 products, conv. filters	-56.2	-64.2	0.843	0.04	3.2	0.3	20
TPU v6e	Factorized BP, three-pass products, fused kernels	-57.5	-64.1	0.886	0.03	3.6	0.2	18
TPU v6e	Factorized BP, single-pass products, fused kernels	-45.8	-54.1	0.705	0.16	10.1	1.0	86
TPU v5e	Factorized BP, three-pass products, fused kernels	-57.6	-64.4	0.886	0.03	3.3	0.2	18
TPU v5e	Factorized BP, three-pass products, fused kernels	-45.8	-54.2	0.705	0.16	9.4	1.0	83
TPU v6e	Factorized BP, six-pass products	-57.5	-64.2	0.886	0.03	3.6	0.2	18
TPU v6e	Factorized BP, three-pass products, direct ramps	-57.5	-64.2	0.884	0.03	3.6	0.2	18
TPU v6e	Factorized BP, single-pass products	-45.8	-54.1	0.707	0.16	9.9	1.0	86
TPU v6e	Factorized BP, single-pass, bfloat16 storage	-45.8	-54.1	0.707	0.16	9.9	1.0	86
TPU v5e	Factorized BP, three-pass products, direct ramps	-57.6	-64.5	0.884	0.03	3.3	0.2	18
TPU v5e	Factorized BP, single-pass products	-45.8	-54.2	0.707	0.16	9.3	1.0	81
CPU	Factorized BP, float64, conv. filters, direct ramps	-59.6	-64.6	0.886	0.03	3.2	0.2	18
CPU	Factorized BP, float32, conv. filters, direct ramps	-58.0	-64.4	0.886	0.03	3.2	0.2	18
TPU v6e	Polar format, float32, corrected	-32.1	-40.1	0.680	0.69	20.7	6.1	70
L4	Polar format, float32, corrected	-32.1	-40.1	0.680	0.69	20.7	6.1	70
CPU	Polar format, float32, gather, corrected	-32.1	-40.2	0.679	0.69	20.1	6.1	70

6.2 Precision

The factorized errors on Melbourne and Iowa are within 1.6 dB of the Panama values of Table 4 (Appendix H). The reference’s own convergence, measured on a crop at 8, 16 and 32 times oversampling, is -69.2 dB at 16 times, so no exact-backprojection error can be measured below about -69 dB. Exact backprojection on the L4 with 8 times oversampled profiles has an error of -56.9 dB with float32 profiles and -56.8 dB with float16; the two coincide because the interpolation error exceeds the arithmetic error at this oversampling. With 16 times profiles the images reach -78.5 and -74.5 dB against the 16 times reference whose interpolation they share, and about -69 dB in truth, at 203.3 s per image with float16 profiles (Appendix F).

Factorized backprojection at float32 accuracy has an error of -57.5 dB on the v6e with three-pass products, -59.5 dB on the L4 and -58.0 dB on the CPU. The float64 image, whose residual is the band limitation of the tiles, is at -59.6 dB; the L4’s float32 kernels come within 0.1 dB of it, the three-pass TPU image and the CPU’s float32 image 1.6 to 2.1 dB above it. On Panama and Melbourne, exact backprojection at 8 times oversampling is therefore no closer to the reference than the L4’s float32 factorized image, and it takes 29 to 60 times as long (float16 and float32 profiles respectively). On the smaller Iowa collection it is 1.5 dB closer. The L4’s image with the float16 tensor-core final stage is 0.2 dB above its float32 image (-59.3 dB); the JAX program’s float16-product image, with every product in float16, was 1.1 dB above that program’s float32 image

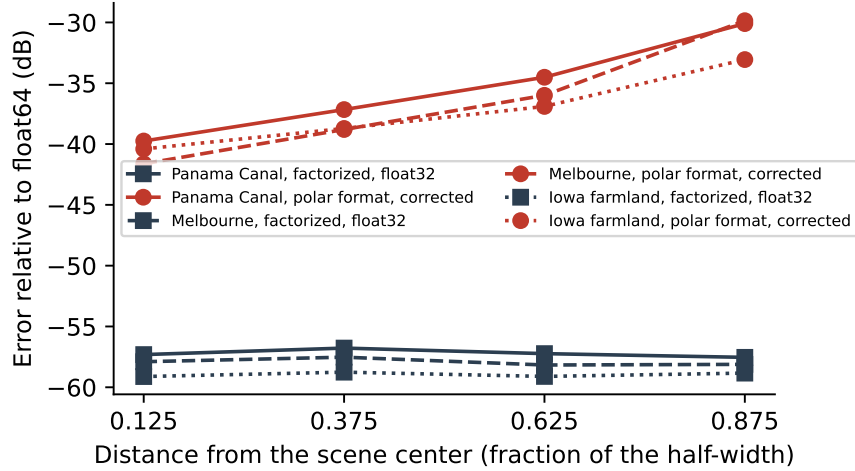


Figure 5: Error relative to the float64 reference in four concentric rings by distance from the scene center, for the corrected polar-format image and the float32 factorized image on the L4, on the three collections.

(−56.2 against −57.3 dB) and its TF32 image 0.5 dB above it (−56.8 dB).

The TPU’s single-pass products round both operands to bfloat16 and give an error of −45.8 dB, which matches the rounding noise of two operands accumulated through three levels (Appendix B). The 99th-percentile amplitude and phase deviations are 0.16 dB and 1.0°; the extremes are 9.9 dB and 86° (172° on Melbourne), at isolated pixels beside bright returns, where Figures 3 and 4 show local losses of coherence. These differences are confined to the complex values and would be expected to matter for interferograms and change-detection maps, which were not tested here.

Three-pass products reproduce the six-pass image to within 0.3 dB in every decibel statistic and 0.003 in coherence. They take 1.4 (v6e) and 1.6 (v5e) times the single-pass time, well below three, because the matrix units are not the limiting unit of the kernels (Appendix D). Bfloat16 storage of the data changes none of these figures and was slower on the two larger collections. The L4’s float16-product image has no TPU equivalent: the TPU converts float16 operands to bfloat16 before the matrix units (Appendix B).

6.3 Polar format

Polar format assumes a planar wavefront, and at these scene sizes the assumption is not good enough on its own. A scatterer at distance ρ from the scene center is focused about $\rho^2/2R$ from where it belongs in range, which at $R = 744$ km (mid-aperture) is 4 m at 2.5 km, some ten pixels. The azimuth displacement is larger and depends on the squint: 7.6 m against 4.3 m at the Panama corners. What saves the algorithm is that the error is almost pure geometry. The fit of Appendix E puts the corner displacement at up to 18 pixels in azimuth and 12 in range, the measured positions of bright targets agree with it to 0.2 pixel, and the image is sharp everywhere. Its 5 by 5 coherence against the reference is only 0.4206, but that is misregistration, not defocus.

The uncorrected image forms in 1.3 to 4.7 s depending on the device (Appendix H), the floor of what polar format can cost. No one can deliver a product whose features are eighteen pixels out of place, so the comparison below is with the corrected form only.

The correction forms the image at twice the pixel density without carriers, resamples it at the

Table 5: Energy per Panama image for the processor alone. The L4 draw is the mean of `nvidia-smi` samples during the timed run; the TPU figures are bounds from third-party design-power estimates, which Google has not published, assuming the chip ran at that power throughout; the CPU figure is the pro-rata share of the 8 cores (25 W of the 400 W package) times the measured utilization. Hosts, memory and the other components of each instance are excluded.

Device	Configuration	Basis	Power (W)	Time (s)	Energy (kJ)
L4	Factorized BP, float16 final, CUDA kernels	measured	65	3.2	0.2
L4	Factorized BP, float32, CUDA kernels	measured	64	3.6	0.2
L4	Exact BP, float16 profiles	measured	71	103.8	7.4
TPU v5e	Factorized BP, single-pass, fused kernels	design-power bound	120 to 200	3.2	0.4 to 0.6
TPU v5e	Factorized BP, three-pass, fused kernels	design-power bound	120 to 200	5.1	0.6 to 1.0
TPU v6e	Factorized BP, single-pass, fused kernels	design-power bound	200 to 350	1.9	0.4 to 0.7
TPU v6e	Factorized BP, three-pass, fused kernels	design-power bound	200 to 350	2.6	0.5 to 0.9
CPU	Factorized BP, float32	pro-rata estimate	25 at 65%	217.4	3.5

apparent positions with a six-tap Lanczos kernel and restores the carriers (Appendix E). It brings the error to -32.1 dB on Panama (-33.4 and -35.3 dB on the other collections), the coherence to 0.9994 and the registration to 0.001 pixel. The residual grows from -40 dB at the center to -30 dB in the outer quarter (Figure 5). Its cause was not identified. Varying the resampling kernel and the raster oversampling changed it by less than 0.6 dB, and longer pulse interpolators made it up to 0.9 dB worse. The registration error and the reference’s convergence are too small to account for it (Appendix E, which also notes one untested candidate).

The correction is an indexed read at a computed position, 36 per pixel. On Panama it adds 0.8 s on the L4 and 6.7 s on the CPU, so the corrected image takes 2.1 s on the L4 and 11.3 s on the CPU, 1.7 and 19 times faster than factorized backprojection on the same device. On the TPUs it takes 97.4 s (v5e) and 105.3 s (v6e), slower than factorized backprojection, because scattered gathers run on the TPU’s vector unit at about 4×10^7 reads per second. For production use on a TPU the correction would have to move to the host or be replaced by an approximation without scattered gathers (Appendix F).

6.4 Power

The L4 forms a float16 factorized Panama image for 0.2 kJ of GPU energy and a float32 image for 0.2 kJ (Table 5). It drew 65 to 72 W in the backprojection loops, close to its 72 W limit, and 55 to 71 W in the polar-format loops. The TPU bounds, 0.4 to 0.6 kJ for a single-pass image on either chip, overlap the L4 figure, so these data do not order the accelerators by energy. The TPU and CPU entries are bounds from unpublished design power and an estimate; none includes the host.

7 Fairness of the comparison

The device comparison depends on each device having been run with an algorithm and arithmetic appropriate to it; Appendices C and F give the measurements behind the following. On the TPU, a profile of the first factorized level shows the two decimation products taking 4.6 ms per tile and the phase rotation before them 9.8 ms, with the rotation moving 534 GB/s. The device time is set by vector-unit work and memory traffic rather than by the matrix units. Padding to multiples of 256 (8.3 to 8.9 s), split-table ramps (4.5 against 3.8 s), bfloat16 storage (5.2 against 4.5 s), block-banded

products and strided convolutions (four times slower) were each tested and none was faster.

On the L4 the exact kernel is dedicated CUDA code with float64 tile geometry. In the JAX program the factorized image used strided convolutions, 1.19 times faster than the dense products with float16 operands and 2.1 times in float32; the L4 rows of Table 3 are the CUDA build. On the CPU a single XLA process reached 65% utilization; four pinned processes raised it to 89% and the throughput by 1.43 times, and float32 (1.6 dB worse than float64 at 2.8 times the speed) is the selected configuration. The factorized flow received a round of kernel work on each accelerator, guided by a stage profile and continued until each stage’s remaining time was set by one unit’s throughput or by the traffic of its own output (Appendix D). The polar-format and exact flows and the CPU code did not receive the same, so their rows are upper bounds on cost. The factorized plan (32-pixel final tiles, three levels, a passband fraction of 0.4 and 70 dB filter attenuation) kept its defaults on every device.

The TPU levels now run at the vector unit’s rate for the rotation arithmetic and the L4 levels within about two times of the memory traffic of their own output. A different factorization (larger final tiles, or the final stage as a chirp-z transform) would move those limits on both devices by an amount this study does not measure. The remaining inequalities are these: the TPU7x and H100 were unavailable under the project’s quota; the polar-format correction was written once, as a gather; the asterisked rows of Table 3 are inverse device times; the TPU hosts (24 and 44 vCPUs) are larger than the work needs while the L4’s (4 vCPUs) is the smallest offered; and spot and committed prices were not considered.

8 Conclusions

The conclusions hold for the fastest form of each flow found in this study (Section 7): kernels written for each accelerator for factorized backprojection, the vendor’s libraries elsewhere.

1. For routine factorized imaging at float32 accuracy, the L4 is the cheapest of the four machines. A thousand Panama images cost \$0.76 on the L4, \$1.79 on the v5e and \$2.09 on the v6e, and the order is the same on Melbourne and Iowa (Table 10). The CPU costs \$40.97 per thousand, 54 times the L4. The one JAX program had put the three accelerators within 11% of one another; the kernels (Appendix D) moved the TPUs by 1.8 times and the L4 by 3.8, and the final ordering follows the size of each device’s kernel gain. The L4’s kernels reach 3.6 s per image, 1.4 times the v6e’s time, at 0.26 of its hourly price.
2. The v6e is the choice when latency matters. It forms the Panama image in 2.6 s with three-pass products, 1.4 times faster than the L4 in float32 at 3.8 times the hourly price, and it is on the Pareto front at no precision. Whatever it forms, the v5e forms at the same error for less, more slowly.
3. Lowering the arithmetic is a different proposition on each device. On the L4 the float16 tensor-core final stage keeps the levels in float32, costs nothing visible (Table 4) and saves 12% of the cost. It is the recommended production setting on that card. The TPUs’ single-pass products cost \$1.15 on the v5e and \$1.54 on the v6e, more than the L4’s group A images. Their images sit 13.6 dB below the L4’s float16 image and 12 dB below the three-pass image of the same device, with phase errors of up to 172° beside bright returns. At that error the TPU image is neither equivalent to the group A images nor cheaper than them. Three-pass products restore float32 accuracy at 1.4 to 1.6 times the single-pass time and are the setting a TPU should be left at for coherent work. Bfloat16 storage of the data gave no advantage.

4. Exact backprojection at 8 times profile oversampling, often taken as the standard, gives no advantage here: on two of three collections it was no more accurate than the float32 factorized image and cost 26 to 56 times as much as that image on the same card. Its accuracy is limited by the profile interpolation, and only at 16 times oversampling does it become the most accurate image in the study (about -69 dB), at 203.3 s per image with float16 profiles.
5. Polar format is the algorithm to use when the budget matters more than the last 25 dB, and it is a GPU algorithm. Corrected, it registers with backprojection and agrees with it to -32.1 dB; on the L4 that image costs \$0.42 per thousand, 1.8 times less than factorized backprojection on the same card after the kernel work (7 times before it), and on the CPU 14 times less. On the TPUs the correction’s scattered reads alone take 42 to 104 s, longer than the whole factorized image, so a TPU should not be used for polar format until the correction is rewritten without gathers or moved to the host. The -32 dB residual is the open problem of this study. Its source was not found and it grows toward the scene edges; it would not trouble an amplitude product, but a user of interferometric or change-detection products should treat -32 dB as a lower bound on the residual until its source is found.
6. The TPU’s levels run at the vector unit’s rate for the phase rotations, and its final stage at the 1/64 utilization that 32 by 32 outputs get from a 256 by 256 systolic array. The L4’s levels are within about two times of the memory traffic of their own output. Neither has a large factor left in this factorization. A larger final tile or a chirp-z final stage would change both limits, and the exact and polar flows and the CPU code have not had a kernel round at all.

Acknowledgments. The code and the text of this report were drafted with the assistance of a large language model; the authors designed the study, ran and checked every measurement, and take full responsibility for the content.

Code and data. The code, the cloud run scripts, every measurement record behind the tables and the build of this report are at <https://github.com/saulpingerman/sar-accel-study>. The radar data are from the Umbra Open Data Program [1] and the optical images from Copernicus Sentinel-2 [3]; neither is redistributed, and the scripts fetch both from their public sources.

A Algorithms

The phase history is motion compensated to the scene reference point. For pulse p with antenna position $\mathbf{a}_p$ and frequency sample f_k ,

$$S_{p,k} = \sum_n \sigma_n \exp\left(-j \frac{4\pi f_k}{c} \Delta R_p(\mathbf{x}_n)\right), \quad \Delta R_p(\mathbf{x}) = \|\mathbf{x} - \mathbf{a}_p\| - \|\mathbf{a}_p\|, \quad (1)$$

where σ_n and $\mathbf{x}_n$ are the complex amplitude and position of scatterer n and c is the speed of light. The CPHD files give the transmit and receive positions per pulse, whose midpoint was used as $\mathbf{a}_p$, and the start frequency and sample spacing per pulse; the mean start frequency and spacing were used, which differ from the per-pulse values by at most 0.02 of a sample spacing. A Taylor weighting ($\bar{n} = 4$, -35 dB sidelobes) [27] was applied along both axes in every algorithm. The image grid is the vendor’s: pixel (i, j) lies at $(i - n_x/2) s_x \mathbf{e}_1 + (j - n_y/2) s_y \mathbf{e}_2$ from the reference point, with $\mathbf{e}_1$ and $\mathbf{e}_2$ the SICD column and row unit vectors and s_x, s_y its sample spacings.

Exact backprojection. Each pulse is range compressed by a zero-padded inverse FFT, with 8 times oversampling in the selected configurations and 16 times in the reference and in the two rows so marked, and each pixel is the sum over pulses of the linearly interpolated profile at ΔR_p times the carrier phase $\exp(j4\pi f_c \Delta R_p / c)$ [20]. The range difference is evaluated in the cancellation-free form

$$\Delta R_p(\mathbf{x}) = \frac{q}{1 + \sqrt{1 + q/r_p}}, \quad q = \frac{\|\mathbf{x}\|^2}{r_p} - 2 \hat{\mathbf{u}}_p \cdot \mathbf{x}, \quad (2)$$

with $r_p = \|\mathbf{a}_p\|$ and $\hat{\mathbf{u}}_p = \mathbf{a}_p / r_p$. The reference implementation evaluates everything in float64 with Numba [32] on the CPU. The CUDA kernel, compiled through CuPy [33], evaluates the base range, the unit vector and the reciprocal range of each 32 by 32 pixel tile in float64 once per pulse, and the per-pixel remainder relative to the tile center in float32; the pulses are processed in chunks of 256 so that the range profiles of one chunk fit in memory at any image size. In the first round the per-pixel geometry was computed in float32 from the scene center and its error grew with the size of the scene; computed from the tile center the error does not grow.

Factorized backprojection. The image is divided recursively into tiles, as in the quadtree [21, 22] and fast factorized [23] backprojection algorithms. At each level the phase history of a parent tile is multiplied by $\exp\{j4\pi f_k(\|\mathbf{c} - \mathbf{a}_p\| - \|\mathbf{c}' - \mathbf{a}_p\|)/c\}$ to move its reference point from the parent center $\mathbf{c}'$ to a child center $\mathbf{c}$; the child’s signal is then band limited in frequency and in pulse index in proportion to the child’s size, and both axes are low-pass filtered and decimated by a Kaiser-windowed sinc [28] with 70 dB design attenuation. At the last level the range to each pixel is linearized about the tile center, the kernel separates in the two image coordinates, and the tile image is a product of two matrices followed by a per-pixel quadratic phase correction, so that no stage requires an indexed read. The implementation handles rectangular images of any size: the side of each level divides the padded image by factors chosen from 8, 7, 6, 5, 4, 3 and 2, with 32 by 32 pixel final tiles, and the decimation factors follow from the range and Doppler extents of a child tile. For the Panama image the three levels divide the azimuth side by 8, 8 and 6 and the range side by 8, 7 and 5, and the first level decimates the 14,399 samples by 6 and the 15,186 pulses by 4. The first two levels compute their phase ramps from float64 geometry on the host and the remaining level from float32 geometry on the device; in the first round a float32 second level was found to add an error that grew with the size of the scene. The ramps themselves are evaluated on the device in float32, either directly (*direct ramps* in the tables) or as the product of a coarse and a fine table (the *split-table* form, about $2\sqrt{n}$ sines and cosines per ramp of n samples, used by every row not labeled *direct ramps*); the device times of both forms are in Appendix H. The decimation filters are applied as dense matrix products (the TPU form), as strided convolutions (the L4 and CPU form) or as explicit sums over taps; on the TPU the three give the same image to better than -140 dB, and on the L4 the convolution differs from the dense product by -64 to -76 dB with TF32 products and not at all with full float32 products. One first-level tile is processed per call to the device, which bounds the working memory to about 6 GB at these sizes on every device. Polar format is described in Appendix E.

B Floating-point formats and the arithmetic of each configuration

Table 6 states, for every configuration that was timed, in which floating-point type each step runs. A first-round configuration in which the phase ramps were also computed in bfloat16 produced an unusable image and is excluded. The L4’s float16-product configuration also holds the data in float16; on the TPU the equivalent change, bfloat16 storage, is timed separately. The gather form

Table 6: Floating-point type of each step in the timed configurations. Geometry means tile centers, ranges and phase-ramp coefficients; ramps means the sines and cosines themselves; products means the operands of the matrix products or convolutions, always accumulated in float32 or better.

Device	Algorithm, label	Data in memory	Geometry	Ramps	Products
L4	Exact BP, float32 profiles	float32 profiles	float64 per tile	float32	float32 sums
L4	Exact BP, float16 profiles	float16 profiles	float64 per tile	float32	float32 sums
L4	Factorized, float32, CUDA kernels	float32	float64 (2 levels), float32	float32	float32 fused multiply-adds (levels and final)
L4	Factorized, float32 levels, float16 final, CUDA kernels	float32	float64 (2 levels), float32	float32	float32 levels; float16 operands on the tensor cores in the final stage
L4	Factorized, float16 products	float16	float64 (2 levels), float32	float32	float16 operands, tensor cores
L4	Factorized, float32	float32	float64 (2 levels), float32	float32	float32 (highest setting)
L4	Factorized, TF32 products	float32	float64 (2 levels), float32	float32	TF32 operands (default setting)
TPU	Factorized, single-pass, fused kernels	float32	float64 (2 levels), float32	float32	bfloat16 operands, 1 pass (levels and final)
TPU	Factorized, three-pass, fused kernels	float32	float64 (2 levels), float32	float32	3 bfloat16 passes (levels and final)
TPU	Factorized, single-pass	float32	float64 (2 levels), float32	float32	bfloat16 operands, 1 pass
TPU	Factorized, three-pass	float32	float64 (2 levels), float32	float32	3 bfloat16 passes
TPU	Factorized, six-pass	float32	float64 (2 levels), float32	float32	6 bfloat16 passes
TPU	Factorized, bfloat16 storage	bfloat16	float64 (2 levels), float32	float32	bfloat16 operands, 1 pass
TPU	Polar format, float32	float32	float64	float32	resampling product 6 passes; float32 FFTs
TPU	Polar format, single-pass product	float32	float64	float32	resampling product 1 pass; float32 FFTs
L4, CPU	Polar format, float32	float32	float64	float32	float32 products and FFTs
CPU	Factorized, float32	float32	float64 (2 levels), float32	float32	float32
CPU	Factorized, float64	float64	float64	float64	float64

Table 7: Floating-point formats. Significant bits include the implicit bit; the spacing is the distance between adjacent representable values near 1.

Format	Bits	Significant bits	Exponent bits	Spacing near 1
float64	64	53	11	2.2×10^{-16}
float32	32	24	8	1.2×10^{-7}
TF32 (GPU product operand)	19	11	8	9.8×10^{-4}
float16 (IEEE binary16)	16	11	5	9.8×10^{-4}
bfloat16	16	8	8	7.8×10^{-3}

of polar format is the selected form on the L4, the CPU and the v5e, and on the v5e the only form that fits, because the dense 15,186 by 15,186 resampling matrix and its intermediates exceed that device’s 16 GB; it has no resampling product, and its arithmetic is that of the L4 and CPU row.

Table 7 lists the formats that appear in this report. Float16 (IEEE binary16) keeps 11 significant bits and a 5-bit exponent, so it resolves 1 part in 2048 but overflows above 65,504; the phase history is therefore scaled to unit peak before conversion. Bfloat16 [30] keeps the 8-bit exponent of float32 and only 8 significant bits, 1 part in 256. On the TPU every matrix product is formed from bfloat16 operands. A float32 operand is either rounded once (one pass), or split into a bfloat16 head and tail with the cross terms accumulated (three passes), or split into three parts (six passes), which recovers float32 accuracy [31]. The L4’s tensor cores accept float16 and bfloat16 operands, and TF32 (11 significant bits) for float32 operands at JAX’s default precision (Section 4). The CPU has no reduced-precision arithmetic that is faster than float32.

The measured image errors agree with the operand resolutions of Table 7. Rounding each operand of the factorized products to bfloat16 adds noise about 48 dB below the operand [29], and the measured error of the single-pass TPU image, once the algorithm’s own error (the float64 row of Table 4) is removed, matches the rounding noise of the two operands accumulated through three levels. The float16 operands of the L4 are 18 dB finer and their rounding adds 1.1 dB to the float32 error. The three-pass product keeps the first 16 significant bits of each operand.

C Using the TPU

Survey. In the first round the gather that backprojection needs per pixel and pulse was measured on the TPU at about 10^8 reads per second; the L4’s CUDA kernel performs the 1.6 trillion reads of the Panama image in 104 s, about 1.5×10^{10} per second. The public documentation of the TPU generations, the JAX and Pallas documentation and the published work on irregular computation on TPUs were searched in October 2026 for a way to run exact backprojection, or any lookup-based formulation, at a rate comparable with the L4. A TPU core has a matrix unit, which multiplies dense tiles of bfloat16 operands at the chip’s rated rate, and a vector unit, which executes element-wise operations and gathers at a rate set by memory bandwidth and lane count, two orders of magnitude lower per element. No texture or cache hierarchy for scattered reads is documented. The v6e and TPU7x add sparse cores for embedding lookups, which Google’s Cloud TPU documentation (accessed October 2026) places near 10^9 rows per second per chip and which are programmed through Pallas, JAX’s kernel language; at that rate the 1.6 trillion reads of the Panama image would take about 30 minutes, against 100 to 200 seconds on the L4. Writing the lookup as a one-hot matrix product, so that the matrix unit does it, multiplies by a matrix whose density is one in several thousand, and the arithmetic cost exceeds the dense algorithm’s by the same factor. No published work was found that forms SAR, beamforming or tomographic images on a TPU; the nearest are formulations of FFTs and non-uniform FFTs as matrix products [34], the approach taken here. The documentation’s recommendations for dense workloads were applied and tested: shapes padded to multiples of 128 (v5e) or 256 (v6e), bfloat16 operands for default products, explicit precision per product and profiling of the compiled program; the recommendation for the TPU7x (two devices per chip) could not be tested.

Profile. Figure 6 shows the time of each stage of the first level on one first-level tile of the Panama collection, 15,186 pulses by 14,399 samples decimated to 3,805 by 2,408, measured one stage at a time on the v6e with single-pass products and on the L4 with TF32 products (profiling one stage at a time materializes the inputs and outputs of each stage, which the fused program does not; for a whole tile these exceed the v5e’s 16 GB, so the v5e was not profiled). The phase ramp fused with the complex multiplication (rotate) costs 9.8 ms on the v6e; the frequency-axis decimation as a dense product 3.5 ms, as a strided convolution 16.4 ms and as a block-banded product 10.9 ms; the pulse-axis decimation 1.0, 12.2 and 3.0 ms. The dense products run at 584 TFLOP/s including the multiplications by the zeros of the filter matrices, and the rotation moves 534 GB/s through memory. On the L4 the same stages take 22.7 ms for the rotation and 83.9, 23.1 and 43.6 ms for the three forms of the frequency-axis decimation, so the strided convolution was used on the L4 and the dense product on the TPU. The deeper levels of the factorization multiply the number of children faster than the decimation shrinks the data, so the element-wise rotation accounts for most of the TPU device time over the whole image; the matrix units are active for roughly a third of the first level and for less of the rest.

Measures that did not reduce the JAX program’s TPU time. (The kernels of Appendix D supersede this program on the TPUs; the figures here describe it.) Padding the pulse and sample counts to multiples of 256 increased the v5e device time from 8.3 to 8.9 s (both timed in one session, earlier than the 8.0 s of Table 11); the padded samples are carried through every level. Bfloat16 storage slowed both TPUs on the Panama and Melbourne images (5.2 compared with 4.5 s on the v6e), because the ramps remain float32 and the resulting conversions take longer than the reduction in memory traffic recovers; on the smaller Iowa image the difference was about 10% in either direction. The split-table ramps, which cut the number of transcendental evaluations by a

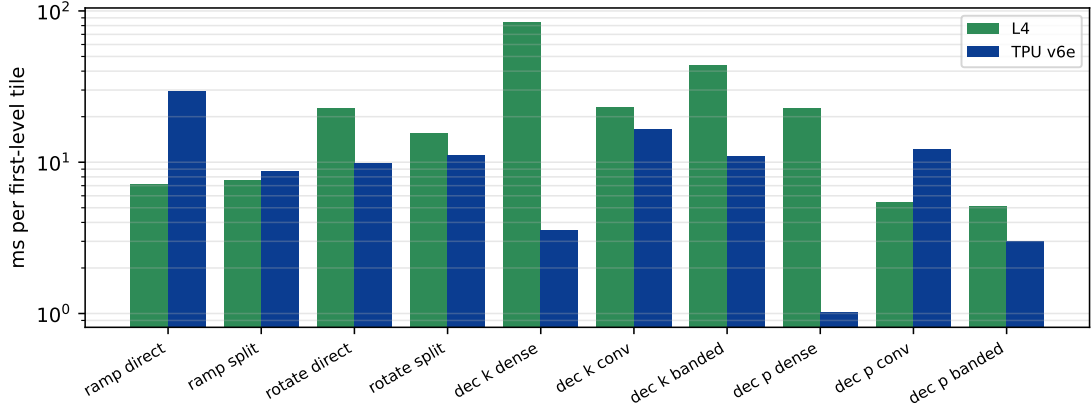


Figure 6: Time per first-level tile of each stage of the factorized algorithm, by form, on the TPU v6e (single-pass products) and the L4 (TF32 products), with float32 data. Logarithmic scale.

factor of about 60, were slower than direct evaluation on both TPUs (4.5 compared with 3.8 s on the v6e). The compiler fuses the direct form into the multiplication, whereas the table products require a pass over memory of their own. At the stage level the strided convolution was five times slower than the dense product on the frequency axis and twelve times on the pulse axis (Figure 6); at the image level the factor was about four for Panama and Melbourne and two to three for Iowa on both TPUs (Appendix H).

Host transfers. A 1.75 GB collection moves from host memory to the v6e at 14 GB/s as two float32 planes, and an order of magnitude more slowly as a complex64 array, which passes through a conversion; the same holds for the image on the return path. The pipelined measurements move planes in both directions; before this change, the polar-format throughput on the v6e corresponded to 3.0 s per image against a device time of 1.3 s.

D Kernels for the factorized algorithm

The factorized rows of Table 3 are formed by kernels written for each accelerator after a stage-by-stage profile of the JAX program, that is, of the code XLA (JAX’s compiler) generates from it; this appendix records the diagnosis, the kernels and the time each step removed (Table 8). Terms from Appendix A: a level splits each parent tile’s phase history into those of its children, the tiles of the next level; level 0 is the split of the whole collection into the first-level tiles, and the two levels after it are the device levels, whose geometry is computed on the device. The XLA program is the baseline. The plan, filters, geometry and arithmetic order are the same in every build, and each build’s image was checked against the dense image at the same precision on a simulated scene before it was timed: single-pass and float16 builds agree with the dense image to the level of their own rounding, three-pass and float32 builds to below -80 dB.

Diagnosis. On the v6e, one first-level tile of the Panama image took 51 ms through the XLA program with single-pass products: 8.3 ms in the first level, 13.3 in the second, 7.9 in the third and 21.4 in the final stage. The three levels are bound by memory traffic: XLA writes the rotated phase history of every child (24 bytes per sample, read back by the decimation product) at about

580 GB/s, a third of the v6e’s 1,640 GB/s, and the children of the three levels total 63 billion samples per image. The final stage materializes the two ramp matrices of every 32 by 32 tile, $2T$ by $P_f Q_f$ each, through five or six element-wise passes before the one product that uses them, and its direct form evaluates 400,000 sines and cosines per tile. The same structure holds on the L4 at the convolution path’s lower bandwidth.

TPU kernels (Pallas). Two kernels, written in Pallas (JAX’s kernel language for the TPU), replace the XLA stages. The level kernel takes a block of pulses and a window of columns of a parent into the core’s vector memory and applies the phase ramps of up to eight children in turn. Each ramp is the product of a coarse table, one angle per 128-column chunk, and a fine table, one angle per column within the chunk, precomputed once per pulse and child outside the kernel. The rotated window is multiplied by the banded block of the decimation matrix on the matrix units, in one or three bfloat16 passes; nothing rotated reaches high-bandwidth memory, and the pulse decimation that follows stays a dense product. Children of one parent share each load, the device levels batch the parents of a tile into one launch, and first-level tiles are formed in groups of sixteen so that the phase history itself is read once per group. Where one block of outputs covers a whole row (the device levels), the window is clipped to the data and read once; at level 0 consecutive output blocks overlap by the filter length.

The final kernel forms the $2T \times 2T$ product of Appendix A ($T = 32$ pixels on a side, $P_f \times Q_f$ pulses by frequency samples per tile) on the matrix units, with both ramps generated by a complex recurrence along the frequency index q : for a pixel offset d along an image axis and the component u_p of the unit vector to the antenna along that axis, the ramp $\exp(-j2\pi du_p(a_0 + a_1 q))$, with a_0 and a_1 the cycles per meter at the first frequency sample and per sample step, is XW^q with $X = \exp(-j2\pi du_p a_0)$ and $W = \exp(-j2\pi du_p a_1)$, so one tile needs four tables of sines and cosines of size $T \times P_f$ instead of one sine per sample. A first version of this kernel with one sine per sample was no faster than XLA (Table 8), which placed that stage on the transcendental unit; the recurrence version removed it. Building the level kernel’s fine tables by the same doubling idea in XLA doubled the image time, because each intermediate table of fewer than 128 columns occupies full 128-lane tiles in TPU memory (the rejected row of Table 8).

After these changes the level kernels run at about 130 billion rotated samples per second on the v6e regardless of level, the vector unit’s rate for the rotation arithmetic, so the remaining time in the levels is set by that unit and not by memory, and the matrix units are nearly idle. The final stage’s floor is the shape of the product: 32 by 32 outputs occupy 1/64 of a 256 by 256 systolic array.

L4 kernels (CUDA). On the GPU the natural form of the same two kernels is CUDA, compiled through CuPy. The level kernel loads the window of 32 pulses into shared memory with consecutive threads on consecutive columns, which is the access pattern the memory system serves in full-width transactions, and rotates it in place for each child, the rotation of each thread’s samples generated as a geometric sequence. Each thread then computes two adjacent outputs of one pulse with the 44-tap decimation filter as fused multiply-adds; for this phase consecutive threads are consecutive pulses, so no two threads of a group read the same shared-memory bank, and the outputs are staged through shared memory and written row by row. The pulse-axis filter is tiled the same way. The final kernel builds the two ramp tables of a tile in shared memory by the same recurrence and each thread accumulates a 4 by 2 block of pixels; in the sixteen-bit build the tables are converted to float16 and the $2T \times 2T$ product runs on the tensor cores with float32 accumulation. A Triton version of the level kernel through Pallas, with the 16-column windows its shared-memory limit allows, gained one to four percent at sixteen bits and was slower in float32; the CUDA builds are

Table 8: Device time per Panama image (s) of each build of the factorized algorithm, two precisions per device: sixteen-bit means single-pass products on the TPUs and float16 products (JAX program) or the float16 final stage (CUDA builds) on the L4; float32 class means three-pass products on the TPUs and float32 on the L4. Each row adds to the one above; the last row of each device is the build of record in Table 3.

Device	Build	16-bit (s)	float32 class (s)
TPU v6e	XLA: ramps materialized, dense decimation products, XLA final stage	3.82	4.60
	Fused level-0 kernel, one child per load	4.51	5.25
	Fused level kernels, children share a load, XLA final stage	4.11	4.75
	Same, device level batched over parents	3.07	3.72
	Plus fused final stage, per-sample sines	3.03	3.70
	Plus fused final stage, ramps by recurrence	2.01	2.75
	Plus clipped windows, no halo at levels 1 and 2, precomputed fine tables (final build)	1.90	2.62
	Fine tables by doubling instead (rejected)	4.06	4.78
TPU v5e	XLA: ramps materialized, dense decimation products, XLA final stage	6.49	9.29
	Fused level-0 kernel, one child per load	7.72	10.11
	Fused level kernels, children share a load, XLA final stage	6.78	8.56
	Same, device level batched over parents	5.03	6.93
	Plus fused final stage, per-sample sines	5.13	7.20
	Plus fused final stage, ramps by recurrence	3.41	5.58
	Plus clipped windows, no halo at levels 1 and 2, precomputed fine tables (final build)	3.18	5.09
	Fine tables by doubling instead (rejected)	6.86	8.76
L4	XLA: ramps materialized, cuDNN convolutions, XLA final stage	11.16	13.69
	Triton level-0 kernel (Pallas), 16-column windows	10.73	18.72
	CUDA: shared-memory FIR levels, one pixel per thread final	–	5.62
	Final stage 2 by 2 pixels per thread	–	4.61
	Pulse-major filter threads (conflict-free reads, scattered loads and stores), tiled pulse filter, 4 by 2 final	–	5.45
	Coalesced window loads, children as in-place phase steps; float16 tensor-core final	3.75	4.39
	Rotation as a geometric sequence per thread; interleaved complex tables in the final	3.55	3.98
	Output tiles staged in shared memory for coalesced stores (final build)	3.16	3.59

what Table 3 reports.

The CUDA code went through six builds, five of them profiled (Table 8; the profiles are in the repository). The profile of the second build placed 1.5 s per image in the level kernel, 1.3 in the final stage and 0.5 in the pulse filter. The pulse-major thread mapping of the third build freed the filter reads of bank conflicts but made the global loads and stores of that kernel scattered, and its time doubled; coalesced loads through shared memory, the geometric-sequence rotation and the staged stores brought it to 1.4 s in the fourth to sixth builds. The memory access pattern changed the time by more than the sine count did. The float32 final stage was bound by the latency of its table-building recurrence and by one shared-memory word per fused multiply-add; the 4 by 2 block per thread with interleaved complex tables brought it from 1.3 to 1.0 s. The tensor-core version is 0.4 s faster per image than the float32 one in the whole-image measurement and 0.2 dB less accurate.

E Polar format on an orbital aperture

The implementation consists of three resampling steps and a two-dimensional FFT, each expressed with FFTs, element-wise products or a single matrix product so that the same code runs on all devices. The pulses are first resampled so that the tangent of the look angle in the image plane is uniform in pulse index, which an orbit does not give exactly; this is a 16-tap Kaiser-windowed sinc interpolation along the pulse axis applied as a dense 15,186 by 15,186 matrix (the form selected on the v6e) or as a sum over 16 gathered rows (the form selected on the L4, the CPU and the v5e; both forms were timed on the L4, the v6e and the CPU). Each pulse is then resampled onto a uniform

radial wavenumber grid, and each wavenumber row onto a uniform cross-range grid, by chirp-z transforms [26], which evaluate the band-limited interpolation on a scaled grid with FFTs and chirps; outputs that would wrap around the band are zeroed. The frequency samples are weighted by f_c/f_k before resampling, because backprojection’s uniform weighting of the samples corresponds to a density of $1/k$ in the wavenumber plane (the polar Jacobian) that the uniform spectral weighting of polar format omits. Without this weighting the corrected image’s error worsens by about 1.5 dB. It is applied in every reported polar-format image.

Because the grazing angle changes along the aperture, the radial band of each pulse projected onto the image plane drifts from pulse to pulse. The raster covers the union of the bands and so keeps the full spectral support of backprojection, a trapezoid, rather than the inscribed rectangle; in the ground plane the rectangle would hold half the bandwidth, in the vendor’s slant plane nearly all of it. The wavenumber grid spacings are set by the native pixel spacings and FFT sizes of 13,824 by 10,800, which include a 300 m guard band on every side of the scene; the range and azimuth profiles are tapered to zero within the guard, because the periodic transforms otherwise wrap echoes from beyond the image into its edges (without the guard the edge error was -5 dB). The image is cut from the FFT output on the native grid with the carrier phases restored.

Distortion and its correction. The planar model writes the range history of a scatterer at $\mathbf{p}$ as $-\hat{\mathbf{u}}_p \cdot \mathbf{q} + c$; a least-squares fit of $\mathbf{q}$ and c over the aperture to the exact history $\|\mathbf{p} - \mathbf{a}_p\| - \|\mathbf{a}_p\|$ gives the position at which polar format focuses the scatterer, $(q_x, q_y + c)$, since a constant range offset is a range shift. The fit residual is below 1 mrad of phase everywhere in the three scenes, so the uncorrected image is displaced without defocus, by up to 18 pixels in azimuth and 12 in range at the corners of the Panama scene; measured target positions agree with the model to 0.2 pixel. The mapping is fitted by cubic polynomials over the image (residual below 10^{-4} pixel).

The image is resampled through the mapping: the spectrum is placed in a raster of twice the pixel density, transformed without the carrier terms, interpolated at the apparent positions with a six-tap Lanczos kernel along each axis, and multiplied by the carriers evaluated at the apparent positions. No separate phase correction is needed; the quadratic phase of about 12 radians at 211 m seen in the uncorrected center crop is the carrier of the range displacement. The oversampled raster is processed in blocks rather than held in memory. The range transform is done once, and the azimuth transform and the resampling proceed in blocks of 512 output range columns.

The corrected image’s error against the reference is -39 dB in the central 1024 by 1024 pixels and -32 dB over the whole Panama image, growing toward the edges (the rings of Table 9 and Figure 5 are quarters of the half-width, not this crop). On the full Panama image the resampling kernel (4, 6 and 8 taps) and the oversampling of the intermediate raster (2 and 3 times) change the whole-image error by less than 0.6 dB and the outermost ring, where the displacement is 10 to 18 pixels, by less than 0.4 dB (Table 9); lengthening the pulse interpolator from 16 to 32 or 48 taps, with a narrower Kaiser window, made the error slightly worse. The registration, measured from the cross-spectrum phase, is 0.001 pixel, and the reference’s own convergence is more than 30 dB below the residual.

A remaining candidate, not tested, is the Taylor weighting, which backprojection applies per pulse and per sample but which polar format carries through the resampling onto the rectangular raster; the resulting difference would increase with distance from the center, as the residual does.

Table 9: Panama Canal, corrected polar format on the CPU: whole-image error and error by ring (dB) for variations of the correction. Rings are the four concentric regions of Appendix H, inner to outer; the selected configuration is the first row.

Variation	Error	Ring 1	Ring 2	Ring 3	Ring 4
6-tap kernel, 2 times oversampled (selected)	−32.1	−39.8	−37.2	−34.5	−30.1
4-tap kernel	−31.6	−37.1	−35.1	−33.5	−29.8
8-tap kernel	−32.2	−40.1	−37.4	−34.7	−30.2
3 times oversampled	−32.2	−40.1	−37.3	−34.7	−30.1
32-tap pulse interpolator, Kaiser 12	−31.7	−37.7	−35.7	−33.8	−29.8
48-tap pulse interpolator, Kaiser 12	−31.2	−36.4	−34.5	−33.1	−29.6

F Measurement protocol in detail

Time and cost. Device time is measured from the start of the per-collection host work (for the factorized algorithm the float64 geometry of the first two levels, about 0.5 s) to the completion of the image on the device. Each configuration was compiled and run once and then timed once more; the second run is reported, and a few configurations were timed twice more, with a spread below 4%, so differences of that order between rows are not resolved.

The pipelined loop ran until at least four images had been formed and at least 40 s (polar format) or 60 s (factorized) had elapsed, whichever came later, except that the exact backprojection loops on the L4 ran two or three images (190 to 495 s), the corrected polar format on the v6e two images (up to 215 s) and the CPU factorized loop one image per process (612 s with four processes), and the reported figure is images divided by elapsed time. On the TPUs the pipelined loop overlaps the host work with the device and its throughput slightly exceeds the inverse of the device time; on the L4, whose host has four vCPUs, the factorized loops ran 6 to 11% slower per image than the device time.

The CPU instance’s utilization was recorded from `/proc/stat` during every run, and its throughput was also measured with two and four worker processes pinned to disjoint cores, because a single XLA process does not occupy all sixteen hardware threads.

Power. The L4’s draw was sampled from `nvidia-smi` twice a second during the timed loops. The TPU virtual machines and the CPU instance expose no power counters; the bounds derived for them from published design figures are in Section 6.4 and are marked as estimates.

Precision. The gain fitted per algorithm family is a complex gain for the backprojections and a real one for polar format, whose phase differs from the reference by a smooth screen before correction. Six statistics are computed for each image. The error is the energy of the difference relative to the energy of the reference (dB); this is the figure quoted when a single number is given. The others are the largest single-pixel difference relative to the brightest pixel (dB); the complex coherence between the two images over a 5 by 5 window, as the mean, the 0.1 percentile and the minimum; the amplitude ratio and the phase difference over the brighter half of the pixels, as the 99th percentile and the maximum; and the error in four rings by distance from the scene center. Appendix H tabulates all of them for every configuration whose image was saved. The selected configurations are defined in Section 5; the implementation forms among which the choice is made are the filter, ramp and resampling forms, and Appendices C and E give the measurements behind each choice.

CPU details. The CPU code is generated by XLA from the same source as the accelerator code and has not been compared with a hand-tuned CPU implementation; the polar-format and float64 rows of Table 3 are single-process timings. Exact backprojection was run on the CPU only as the float64 reference with 16 times oversampling, which took 1892 s on the n2-highmem-48 of Section 4; at about \$1,210 per thousand images it serves only as the reference and is not a candidate configuration.

Reference convergence and exact backprojection. The reference is a float64 exact backprojection with range profiles oversampled 16 times. Its convergence was measured on a 1024 by 1024 crop by forming the same crop at 8, 16 and 32 times: the 8 times image differs from the 32 times image by -55.5 dB and the 16 times image from the 32 times image by -69.2 dB. At 8 times the convergence error would have been comparable to the differences under study, so the 16 times reference was used; even so, no exact-backprojection error can be measured below the convergence of the reference.

With 16 times oversampled profiles, against the 16 times reference whose interpolation they share, the L4 float32 and float16 images measure -78.5 and -74.5 dB. At these settings the error of exact backprojection is set by the profile oversampling, approximately -56 dB at 8 times and, within the limit set by the convergence of the reference itself, approximately -69 dB at 16 times; float16 storage of the profiles adds no measurable error at 8 times, and at 16 times it separates the two shared-interpolation figures by 4 dB, which against the -69 dB floor amounts to less than 1 dB. With 16 times oversampling, exact backprojection takes 203.3 s with float16 profiles and 295.2 s with float32 profiles, which at the instance price is \$39.9 and \$58.0 per thousand images.

Polar format on each device. On the v5e the correction adds 94.3 s to the 3.1 s image. The contiguous row gathers of the pulse resampling run nearly a hundred times faster on the TPU than the scattered two-dimensional gathers of the correction. A separable approximation of the correction, expressible as row and column phase ramps, would avoid the gathers, but the squinted geometry of these collections is not exactly separable in this way. A form of the correction that avoids scattered gathers would reduce the TPU’s disadvantage in Section 6.3; the size of the reduction was not measured. The dense pulse-resampling product (images saved on the L4, the v6e and the CPU) and the gather form (the v5e and the CPU) give images that agree to the precision of Table 4.

G Ground truth of the Panama regions

The identity of the Panama regions used in Figure 3 was checked against Copernicus Sentinel-2 imagery [3] (Figure 8). Each pixel of the images formed here was located on the ground from the vendor’s SICD header: the pixel’s position in the slant plane follows from the scene center point and the row and column unit vectors, and the ground point is the intersection of the line through it along the slant plane’s normal with the WGS84 ellipsoid at the scene height. The image corners so computed agree with the corner coordinates in the SICD header to within 15 m. The nearest usable Sentinel-2 acquisition is from 2023-07-23, five days later, at 10 m resolution; a dry-season acquisition of 2024-02-23 is shown as well because clouds cover part of the earlier one. The lock chambers, the quays and container yard of the terminal, and the road beside the vegetation region coincide in the two sensors. The bright target of the ship region, about 180 m long and close to the eastern bank, is not a fixed structure: the Panama Canal is a repeat-collection site in the Umbra catalog, and in the vendor’s geocoded products of six other passes, four of them three, four, five and seven days after this collection and two in February 2024, the same water is empty (Figure 9). The pass of

2023-07-25 02:30 UTC repeats the geometry of this collection to within 0.3° (azimuth 122.4° against 122.1° , grazing angle 42.6° in both, left-looking), which rules out an aspect-dependent return from a fixed object; the other passes span azimuths from -72° to 223° and grazing angles from 40° to 63° . The pass of 2023-07-23 14:47 UTC precedes the Sentinel-2 image of that day by 64 minutes, and neither shows anything at the position. The target is therefore a ship in the navigation channel; the Sentinel-2 image of 2024-02-23 shows a different ship nearby. The vendor’s geocoded product of this collection places the target inside the outlined crop, which confirms the pixel-to-ground mapping independently of the corner check.

The ship appears on the eastern bank, inland of the channel, as a consequence of its motion during the aperture. A stationary-scene processor places a target with line-of-sight velocity v_r at the position whose Doppler history matches, displaced along azimuth by about Rv_r/V (here about 100 m per m/s), and the along-track velocity defocuses it [2]. Since exact backprojection takes the antenna position of every pulse as data, a target moving at constant velocity $\mathbf{v}$ is formed exactly by backprojecting with the track shifted by $-\mathbf{v}t$, which both focuses it and places it at its position at the aperture center. The ship was assumed to follow the channel axis (bearing 158° , from the Sentinel-2 water mask) and its speed was swept from -8 to 8 m/s in a tile around it (Figure 7). The sharpness of the tile, the sum of the squared intensities, is single-peaked at 3.0 m/s (5.8 knots toward the Pacific), where the peak intensity is 10.6 dB above the stationary-scene image and the ship has moved 469 pixels (194 m) along azimuth into the navigation channel; the position of the water was not used in the estimate. A heading 10° to the west (148°) gives the same speed with an 18% lower sharpness peak and a compensated position 22 m away. The images compared in Figures 3 and 4 are the stationary-scene images, in which the ship appears at the same displaced position under every algorithm; the motion analysis serves only to establish that the target is a ship.

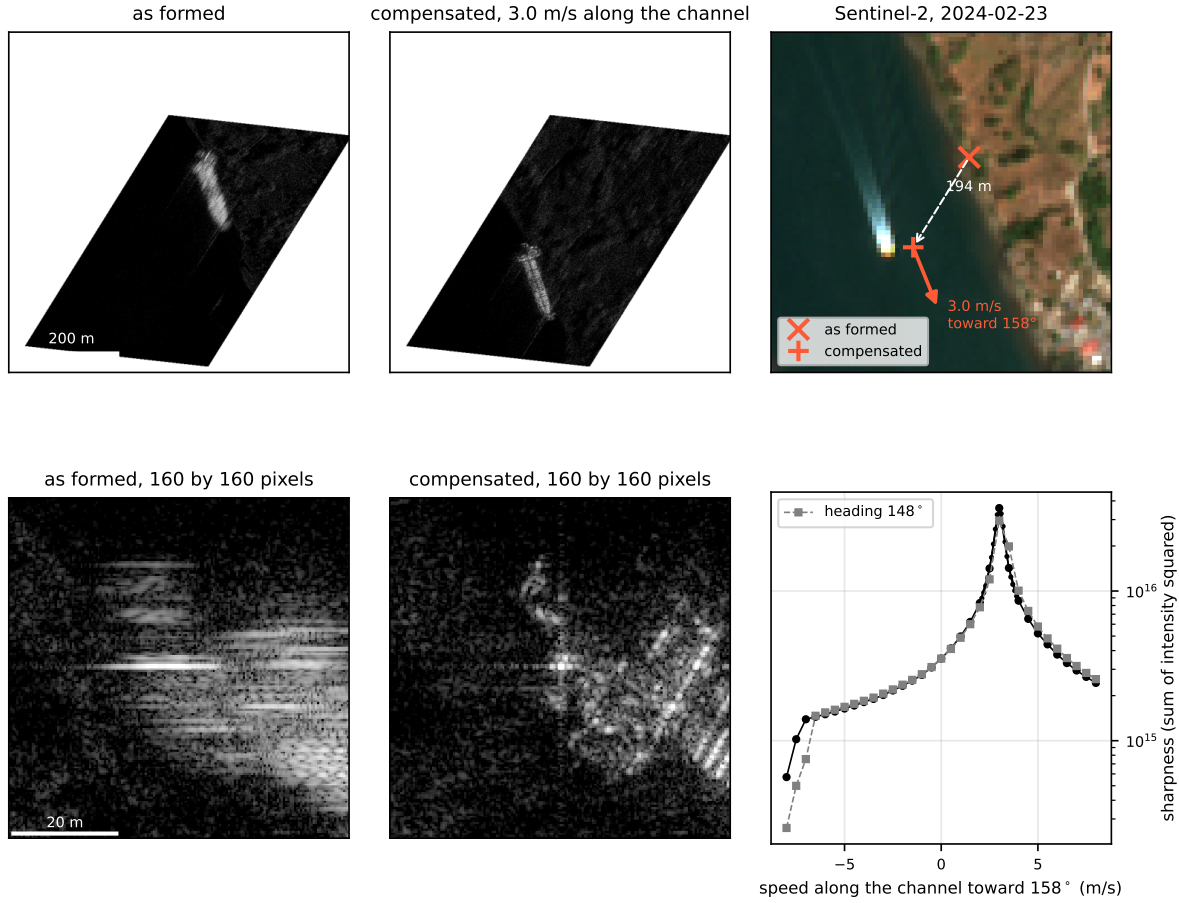


Figure 7: The ship compensated for its motion. Top: the tile around the ship as formed and after backprojection with the antenna track shifted for a ship moving at 3.0 m/s along the channel, both resampled north up over a 760 m window (200 m bar), and the two positions on the Sentinel-2 image of 2024-02-23, with the 194 m displacement (dashed) and the estimated velocity, 3.0 m/s toward 158°, drawn at the compensated position. Bottom: 160 by 160 pixel windows around the ship in image coordinates (20 m bar), and the sharpness of the tile against the assumed speed for the channel heading of 158° (coarse and fine sweeps) and for 148°.

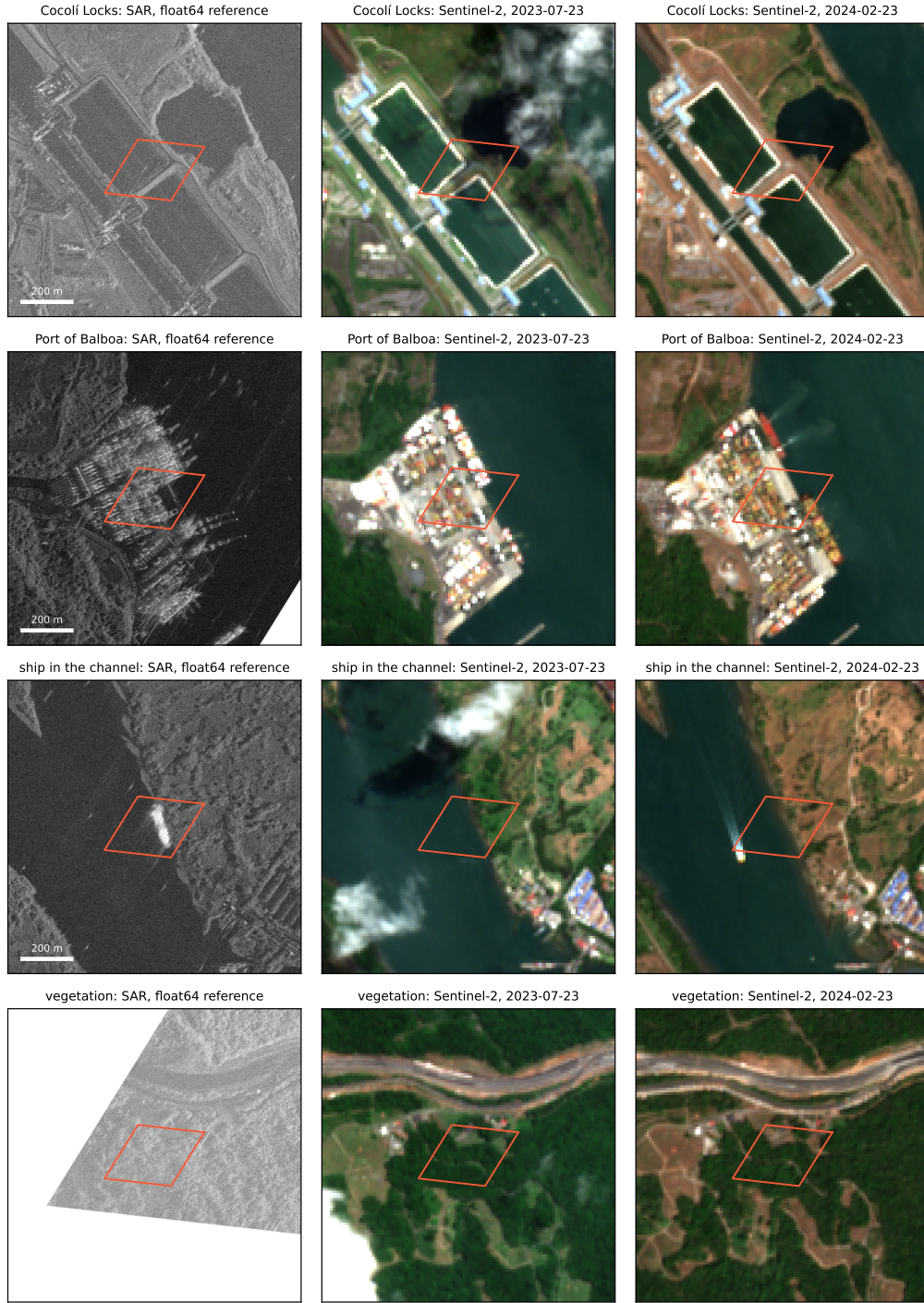


Figure 8: The four Panama regions of Figure 3 located on the ground. Left: the float64 reference image resampled to a north-up 1 m grid over a 1.2 km window (amplitude in decibels over a 45 dB range, 200 m bar). Center and right: Copernicus Sentinel-2 true-color images of the same windows from 2023-07-23, five days after the radar collection, and from 2024-02-23. The outline is the 512 by 512 pixel crop of Figure 3; it is a parallelogram on the ground because the image axes are azimuth and slant range.

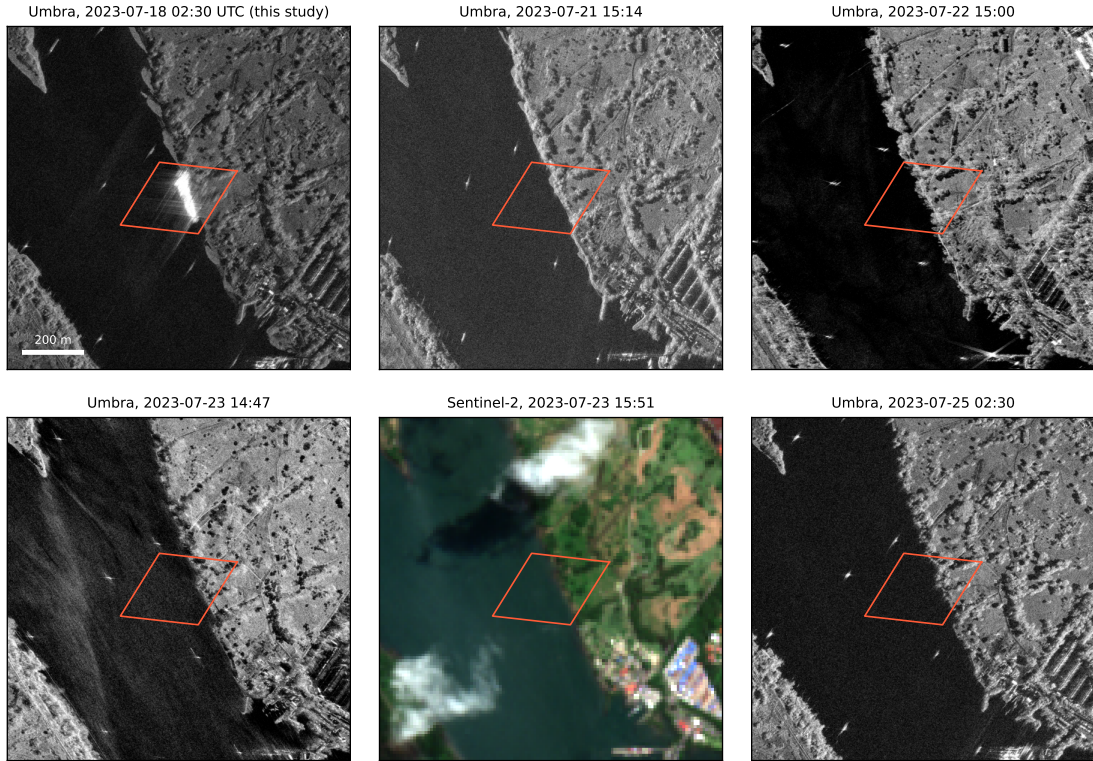


Figure 9: The ship region on six dates, north up, 1.2 km windows (200 m bar): the vendor’s geocoded products of five Umbra passes between 2023-07-18 and 2023-07-25, and the Sentinel-2 image of 2023-07-23. The outline is the crop of Figure 3. The bright target of 2023-07-18 is absent from every other pass, including that of 2023-07-25, which repeats the viewing geometry of 2023-07-18 to within 0.3° ; the smaller bright points along the eastern bank recur in every pass and are consistent with moored small craft and shore structures.

H Full tables

Tables 10 to 13 give the cost figures for Melbourne and Iowa and, for every configuration timed or measured on each scene, the device time and the statistics. The column *Rings* gives the error in four concentric regions, the quarters of the half-width from the scene center outward. Configurations are labeled as in the body: on the TPUs *single-pass*, *three-pass* and *six-pass products* are the one, three and six bfloat16 passes of a float32 product and *bfloat16 storage* keeps the data in bfloat16; on the L4 *TF32 products* is the product at JAX’s default precision and *float32* the product at the highest setting, used for every float32 figure in the body; *gather* marks the gather form of the pulse resampling and of the correction (for polar format in gather form, *single-pass product* denotes the run whose dense product would have been single-pass; the gather form has no such product and the two images are identical); *direct ramps*, *conv. filters* and *tap sums* are the forms described in Appendix A. On the L4, *float16 product* denotes the pulse-resampling product with float16 tensor-core operands. The pipelined figures in parentheses for the polar-format *single-pass product* rows of both TPUs were recorded before the transfer change described in Appendix C and exceed the device time for that reason. The L4 polar-format times for Iowa (1.5 s uncorrected, 1.6 s corrected) differ by less than the correction’s cost on the other scenes, and the CPU’s dense uncorrected Melbourne time (20.3 s) exceeds its corrected time (17.8 s); both anomalies are single runs of short kernels and were not repeated.

Table 10: Melbourne and Iowa: device time, throughput and cost for the selected configurations, as in Table 3.

Device	Algorithm	Arithmetic	Time (s)	Images/h	\$ per 1000
<i>Melbourne</i>					
L4	Exact BP	float16 profiles	102.7	35	19.99
L4	Exact BP	float32 profiles	248.2	15	48.58
L4	Factorized BP	float32 levels, float16 final, CUDA kernels	3.1	1,037	0.68
L4	Factorized BP	float32, CUDA kernels	3.6	911	0.78
L4	Polar format	float32, gather, corrected	2.2	1,562	0.45
TPU v5e	Factorized BP	single-pass products, fused kernels	3.0	1,103	1.09
TPU v5e	Factorized BP	three-pass products, fused kernels	5.2	661	1.82
TPU v5e	Polar format	float32, gather, corrected	62.5	58*	20.84
TPU v6e	Factorized BP	single-pass products, fused kernels	1.7	1,921	1.41
TPU v6e	Factorized BP	three-pass products, fused kernels	2.4	1,402	1.93
TPU v6e	Polar format	float32, corrected	101.7	35	77.96
CPU	Factorized BP	float32, conv. filters, direct ramps	234.7	21	45.95
CPU	Factorized BP	float64, conv. filters, direct ramps	642.0	6*	171.90
CPU	Polar format	float32, gather, corrected	11.6	310*	3.11
<i>Iowa farmland</i>					
L4	Exact BP	float16 profiles	63.8	57	12.46
L4	Exact BP	float32 profiles	142.3	25	27.85
L4	Factorized BP	float32 levels, float16 final, CUDA kernels	2.0	1,632	0.43
L4	Factorized BP	float32, CUDA kernels	2.3	1,424	0.50
L4	Polar format	float32, gather, corrected	1.6	2,108	0.34
TPU v5e	Factorized BP	single-pass products, fused kernels	2.2	1,549	0.77
TPU v5e	Factorized BP	three-pass products, fused kernels	3.5	982	1.22
TPU v5e	Polar format	float32, gather, corrected	44.2	81*	14.74
TPU v6e	Factorized BP	single-pass products, fused kernels	1.4	2,431	1.11
TPU v6e	Factorized BP	three-pass products, fused kernels	1.9	1,790	1.51
TPU v6e	Polar format	float32, corrected	57.8	61	44.49
CPU	Factorized BP	float32, conv. filters, direct ramps	191.0	32	30.27
CPU	Factorized BP	float64, conv. filters, direct ramps	409.9	9*	109.75
CPU	Polar format	float32, gather, corrected	8.2	440*	2.19

Table 11: Every configuration timed or measured, Panama Canal: device time in seconds, with the pipelined seconds per image in parentheses where a loop was run; error and largest pixel difference (dB), 5 by 5 coherence statistics, amplitude and phase statistics over the brighter half of the pixels, and the error in four rings by distance from the scene center (quarters of the half-width, dB). A dash marks a configuration that was timed without saving its image.

Device	Configuration	Time	Error	Max	Coh mean	Coh p0.1	Coh min	Amp p99	Amp max	Ph p99	Ph max	Rings
L4	Exact BP, float16 profiles	103.8 (99.3)	-56.8	-62.2	1.0000	1.000	0.922	0.03	2.5	0.2	15	-57, -56, -57, -57
L4	Exact BP, float16 profiles, 16x oversampled	203.3	-74.5	-85.2	1.0000	1.000	1.000	0.01	0.1	0.0	1	-74, -75, -75, -74
L4	Exact BP, float32 profiles	216.4 (216.4)	-56.9	-62.2	1.0000	1.000	0.923	0.03	2.5	0.2	15	-57, -56, -57, -57
L4	Exact BP, float32 profiles, 16x oversampled	295.2	-78.5	-85.5	1.0000	1.000	1.000	0.00	0.1	0.0	0	-78, -79, -79, -78
L4	Factorized BP, float16 products	13.0	-56.1	-64.9	1.0000	1.000	0.843	0.04	3.2	0.3	20	-56, -56, -56, -56
L4	Factorized BP, float16 products, conv. filters	10.9 (12.0)	-56.2	-64.2	1.0000	1.000	0.843	0.04	3.2	0.3	20	-56, -56, -56, -56
L4	Factorized BP, float16 products, direct ramps	12.8 (13.8)	-	-	-	-	-	-	-	-	-	-
L4	Factorized BP, float32 levels, float16 final, CUDA kernels	3.2 (3.4)	-59.3	-64.4	1.0000	1.000	0.886	0.03	3.2	0.2	18	-59, -59, -59, -60
L4	Factorized BP, float32	29.4	-57.3	-64.3	1.0000	1.000	0.886	0.03	3.4	0.2	18	-57, -57, -57, -58
L4	Factorized BP, float32, conv. filters	13.9 (14.8)	-	-	-	-	-	-	-	-	-	-
L4	Factorized BP, float32, CUDA kernels	3.6 (3.8)	-59.5	-64.7	1.0000	1.000	0.886	0.03	3.2	0.2	18	-59, -59, -59, -60
L4	Factorized BP, TF32 products	19.3	-56.8	-65.0	1.0000	1.000	0.842	0.04	3.2	0.2	20	-57, -56, -57, -57
L4	Factorized BP, TF32 products, conv. filters	13.8 (15.0)	-	-	-	-	-	-	-	-	-	-
L4	Polar format, float16 product	1.6 (1.3)	2.9	-0.0	0.4206	0.013	0.000	22.70	82.9	178.2	180	3, 3, 3, 3
L4	Polar format, float32, corrected	2.3 (2.1)	2.9	-0.0	0.4206	0.013	0.000	22.70	88.0	178.2	180	3, 3, 3, 3
L4	Polar format, float32, gather	3.0 (3.2)	-32.1	-40.1	0.9994	0.986	0.680	0.69	20.7	6.1	70	-40, -37, -35, -30
L4	Polar format, float32, gather, corrected	1.3 (1.6)	-	-	-	-	-	-	-	-	-	-
L4	Polar format, float32, gather, corrected	2.1 (2.1)	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass, bfloat16 storage	12.7	-45.8	-54.2	0.9999	0.998	0.707	0.16	9.3	1.0	81	-45, -45, -46, -46
TPU v5e	Factorized BP, six-pass products	17.6	-57.6	-64.5	1.0000	1.000	0.886	0.03	3.4	0.2	18	-58, -57, -58, -58
TPU v5e	Factorized BP, six-pass products, direct ramps	13.3	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass products	8.0	-45.8	-54.2	0.9999	0.998	0.707	0.16	9.3	1.0	81	-45, -45, -46, -46
TPU v5e	Factorized BP, single-pass products, conv. filters	31.9	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass products, direct ramps	6.5 (6.2)	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass products, fused kernels	3.2 (3.5)	-45.8	-54.2	0.9999	0.998	0.705	0.16	9.4	1.0	83	-45, -45, -46, -46
TPU v5e	Factorized BP, single-pass products, fused kernels, direct-trig final	4.9	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass products, fused kernels, XLA final stage	4.8	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, three-pass products	13.8	-57.6	-64.3	1.0000	1.000	0.884	0.03	3.3	0.2	18	-58, -57, -58, -58
TPU v5e	Factorized BP, three-pass products, direct ramps	9.3 (9.0)	-57.6	-64.5	1.0000	1.000	0.884	0.03	3.3	0.2	18	-58, -57, -58, -58
TPU v5e	Factorized BP, three-pass products, fused kernels	5.1 (5.4)	-57.6	-64.4	1.0000	1.000	0.886	0.03	3.3	0.2	18	-58, -57, -58, -58
TPU v5e	Factorized BP, three-pass products, fused kernels, direct-trig final	6.7	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, three-pass products, fused kernels, XLA final stage	6.4	-	-	-	-	-	-	-	-	-	-
TPU v5e	Polar format, single-pass product, gather	3.1 (4.8)	2.9	-0.0	0.4206	0.013	0.000	22.70	87.9	178.2	180	3, 3, 3, 3
TPU v5e	Polar format, float32, gather	3.1 (3.0)	2.9	-0.0	0.4206	0.013	0.000	22.70	87.9	178.2	180	3, 3, 3, 3
TPU v5e	Polar format, float32, gather, corrected	97.4	-32.1	-40.1	0.9994	0.986	0.680	0.69	20.7	6.1	70	-40, -37, -35, -30
TPU v6e	Factorized BP, single-pass, bfloat16 storage	5.2	-45.8	-54.1	0.9999	0.998	0.707	0.16	9.9	1.0	86	-45, -45, -46, -46
TPU v6e	Factorized BP, six-pass products	7.2	-57.5	-64.2	1.0000	1.000	0.886	0.03	3.6	0.2	18	-58, -57, -57, -58
TPU v6e	Factorized BP, single-pass products, direct ramps	5.9	-	-	-	-	-	-	-	-	-	-
TPU v6e	Factorized BP, single-pass products	4.5	-45.8	-54.1	0.9999	0.998	0.707	0.16	9.9	1.0	86	-45, -45, -46, -46
TPU v6e	Factorized BP, single-pass products, conv. filters	18.6	-	-	-	-	-	-	-	-	-	-
TPU v6e	Factorized BP, single-pass products, direct ramps	3.8 (3.5)	-	-	-	-	-	-	-	-	-	-
TPU v6e	Factorized BP, single-pass products, fused kernels	1.9 (2.1)	-45.8	-54.1	0.9999	0.998	0.705	0.16	10.1	1.0	86	-45, -45, -46, -46
TPU v6e	Factorized BP, single-pass products, fused kernels, direct-trig final	2.9	-	-	-	-	-	-	-	-	-	-
TPU v6e	Factorized BP, single-pass products, fused kernels, XLA final stage	3.0	-	-	-	-	-	-	-	-	-	-
TPU v6e	Factorized BP, three-pass products	6.1	-57.5	-64.0	1.0000	1.000	0.884	0.03	3.6	0.2	18	-58, -57, -57, -58
TPU v6e	Factorized BP, three-pass products, direct ramps	4.6 (4.3)	-57.5	-64.2	1.0000	1.000	0.884	0.03	3.6	0.2	18	-58, -57, -57, -58
TPU v6e	Factorized BP, three-pass products, fused kernels	2.6 (2.8)	-57.5	-64.1	1.0000	1.000	0.886	0.03	3.6	0.2	18	-58, -57, -57, -58
TPU v6e	Factorized BP, three-pass products, fused kernels, direct-trig final	3.6	-	-	-	-	-	-	-	-	-	-
TPU v6e	Factorized BP, three-pass products, fused kernels, XLA final stage	3.6	-	-	-	-	-	-	-	-	-	-
TPU v6e	Polar format, float32	1.3 (1.3)	2.9	-0.0	0.4206	0.013	0.000	22.70	88.1	178.2	180	3, 3, 3, 3
TPU v6e	Polar format, float32, corrected	105.3 (107.5)	-32.1	-40.1	0.9994	0.986	0.680	0.69	20.7	6.1	70	-40, -37, -35, -30
TPU v6e	Polar format, single-pass product	1.3 (3.0)	2.9	-0.0	0.4206	0.013	0.000	22.70	82.3	178.2	180	3, 3, 3, 3
TPU v6e	Polar format, float32, gather	1.4	-	-	-	-	-	-	-	-	-	-
TPU v6e	Polar format, float32, gather, corrected	105.4	-	-	-	-	-	-	-	-	-	-
CPU	Factorized BP, float32, conv. filters, direct ramps	217.4	-58.0	-64.4	1.0000	1.000	0.886	0.03	3.2	0.2	18	-58, -58, -58, -58
CPU	Factorized BP, float64, conv. filters, direct ramps	616.3	-59.6	-64.6	1.0000	1.000	0.886	0.03	3.2	0.2	18	-59, -59, -59, -60
CPU	Polar format, float32	12.5	2.9	-0.0	0.4206	0.013	0.000	22.70	80.2	178.2	180	3, 3, 3, 3
CPU	Polar format, float32, corrected	17.0	-	-	-	-	-	-	-	-	-	-
CPU	Polar format, float32, gather	4.7	-	-	-	-	-	-	-	-	-	-
CPU	Polar format, float32, gather, corrected	11.3	-32.1	-40.2	0.9994	0.986	0.679	0.69	20.1	6.1	70	-40, -37, -35, -30

Table 12: Every configuration timed or measured, Melbourne: device time in seconds, with the pipelined seconds per image in parentheses where a loop was run; error and largest pixel difference (dB), 5 by 5 coherence statistics, amplitude and phase statistics over the brighter half of the pixels, and the error in four rings by distance from the scene center (quarters of the half-width, dB). A dash marks a configuration that was timed without saving its image.

Device	Configuration	Time	Error	Max	Coh mean	Coh p0.1	Coh min	Amp p99	Amp max	Ph p99	Ph max	Rings
L4	Exact BP, float16 profiles	102.7 (101.8)	-56.8	-61.3	1.0000	1.000	0.957	0.04	8.2	0.3	22	-57, -57, -57, -57
L4	Exact BP, float32 profiles	248.2 (247.4)	-56.8	-61.3	1.0000	1.000	0.958	0.04	8.2	0.3	22	-57, -57, -57, -57
L4	Factorized BP, float16 products	14.8	-56.5	-57.3	1.0000	1.000	0.929	0.05	2.8	0.3	24	-57, -56, -57, -57
L4	Factorized BP, float16 products, conv. filters	12.1 (13.1)	-56.5	-57.5	1.0000	1.000	0.929	0.05	2.8	0.3	24	-57, -56, -57, -57
L4	Factorized BP, float16 products, direct ramps	14.2	-	-	-	-	-	-	-	-	-	-
L4	Factorized BP, float32 levels, float16 final, CUDA kernels	3.1 (3.5)	-58.9	-58.5	1.0000	1.000	0.947	0.04	2.4	0.2	21	-59, -58, -59, -59
L4	Factorized BP, float32	33.4	-58.0	-57.4	1.0000	1.000	0.947	0.04	2.4	0.3	21	-58, -58, -58, -58
L4	Factorized BP, float32, conv. filters	15.4 (16.3)	-	-	-	-	-	-	-	-	-	-
L4	Factorized BP, float32, CUDA kernels	3.6 (4.0)	-59.1	-58.2	1.0000	1.000	0.947	0.04	2.4	0.2	21	-59, -59, -59, -59
L4	Factorized BP, TF32 products	22.4	-57.2	-57.4	1.0000	1.000	0.928	0.05	2.8	0.3	24	-57, -57, -57, -57
L4	Factorized BP, TF32 products, conv. filters	15.1 (15.9)	-	-	-	-	-	-	-	-	-	-
L4	Polar format, float16 product	1.7 (1.4)	2.5	-4.8	0.4401	0.013	0.000	23.16	85.1	178.2	180	1, 3, 3, 3
L4	Polar format, float32	2.4 (2.2)	2.5	-4.8	0.4401	0.013	0.000	23.16	81.8	178.2	180	1, 3, 3, 3
L4	Polar format, float32, corrected	3.2 (3.3)	-33.4	-46.3	0.9997	0.994	0.873	0.52	23.9	4.1	180	-42, -39, -36, -30
L4	Polar format, float32, gather	1.4 (1.7)	-	-	-	-	-	-	-	-	-	-
L4	Polar format, float32, gather, corrected	2.2 (2.3)	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass, bfloat16 storage	10.7	-46.8	-54.5	1.0000	0.999	0.858	0.17	12.9	1.1	172	-48, -46, -47, -47
TPU v5e	Factorized BP, six-pass products	17.6	-57.7	-57.3	1.0000	1.000	0.947	0.04	2.4	0.3	21	-58, -57, -58, -58
TPU v5e	Factorized BP, six-pass products, direct ramps	14.7	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass products	7.9	-46.8	-54.5	1.0000	0.999	0.858	0.17	12.9	1.1	172	-48, -46, -47, -47
TPU v5e	Factorized BP, single-pass products, conv. filters	32.2	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass products, direct ramps	6.8 (6.5)	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass products, fused kernels	3.0 (3.3)	-46.8	-54.5	1.0000	0.999	0.859	0.17	13.1	1.1	171	-48, -46, -47, -47
TPU v5e	Factorized BP, three-pass products	13.1	-57.7	-57.6	1.0000	1.000	0.946	0.04	2.5	0.3	21	-58, -57, -58, -58
TPU v5e	Factorized BP, three-pass products, direct ramps	10.0 (9.7)	-57.7	-57.3	1.0000	1.000	0.946	0.04	2.5	0.3	21	-58, -57, -58, -58
TPU v5e	Factorized BP, three-pass products, fused kernels	5.2 (5.4)	-57.7	-57.4	1.0000	1.000	0.947	0.04	2.4	0.3	21	-58, -57, -58, -58
TPU v5e	Polar format, single-pass product, gather	3.0 (4.6)	2.5	-4.8	0.4401	0.013	0.000	23.16	81.8	178.2	180	1, 3, 3, 3
TPU v5e	Polar format, float32, gather	3.0 (2.9)	2.5	-4.8	0.4401	0.013	0.000	23.16	81.8	178.2	180	1, 3, 3, 3
TPU v5e	Polar format, float32, gather, corrected	62.5	-33.4	-46.3	0.9997	0.994	0.873	0.52	23.9	4.1	180	-42, -39, -36, -30
TPU v5e	Factorized BP, single-pass, bfloat16 storage	5.6	-46.8	-53.7	1.0000	0.999	0.859	0.17	13.1	1.1	172	-47, -46, -47, -47
TPU v6e	Factorized BP, six-pass products	7.5	-57.5	-55.9	1.0000	1.000	0.947	0.04	2.4	0.3	21	-57, -57, -58, -58
TPU v6e	Factorized BP, six-pass products, direct ramps	6.3	-	-	-	-	-	-	-	-	-	-
TPU v6e	Factorized BP, single-pass products	4.4	-46.8	-53.7	1.0000	0.999	0.859	0.17	13.2	1.1	172	-47, -46, -47, -47
TPU v6e	Factorized BP, single-pass products, conv. filters	18.0	-	-	-	-	-	-	-	-	-	-
TPU v6e	Factorized BP, single-pass products, direct ramps	3.9 (3.6)	-	-	-	-	-	-	-	-	-	-
TPU v6e	Factorized BP, single-pass products, fused kernels	1.7 (1.9)	-46.8	-53.8	1.0000	0.999	0.857	0.17	13.2	1.1	172	-48, -46, -47, -47
TPU v6e	Factorized BP, three-pass products	6.0	-57.4	-56.1	1.0000	1.000	0.946	0.04	2.5	0.3	21	-57, -57, -58, -58
TPU v6e	Factorized BP, three-pass products, direct ramps	4.6 (4.4)	-57.5	-55.9	1.0000	1.000	0.946	0.04	2.5	0.3	21	-57, -57, -58, -58
TPU v6e	Factorized BP, three-pass products, fused kernels	2.4 (2.6)	-57.5	-56.0	1.0000	1.000	0.947	0.04	2.4	0.3	21	-57, -57, -58, -58
TPU v6e	Polar format, float32	1.4 (1.3)	2.5	-4.8	0.4401	0.013	0.000	23.16	81.8	178.2	180	1, 3, 3, 3
TPU v6e	Polar format, float32, corrected	101.7 (104.0)	-33.4	-46.3	0.9997	0.994	0.873	0.52	23.9	4.1	180	-42, -39, -36, -30
TPU v6e	Polar format, single-pass product	1.2 (2.9)	2.5	-4.8	0.4401	0.013	0.000	23.16	89.8	178.2	180	1, 3, 3, 3
TPU v6e	Polar format, float32, gather	1.3	-	-	-	-	-	-	-	-	-	-
TPU v6e	Polar format, float32, gather, corrected	101.8	-	-	-	-	-	-	-	-	-	-
CPU	Factorized BP, float32, conv. filters, direct ramps	234.7	-58.0	-56.4	1.0000	1.000	0.947	0.04	2.4	0.3	21	-58, -58, -58, -58
CPU	Factorized BP, float64, conv. filters, direct ramps	642.0	-59.1	-58.3	1.0000	1.000	0.947	0.04	2.4	0.2	21	-59, -59, -59, -59
CPU	Polar format, float32	20.3	2.5	-4.8	0.4401	0.013	0.000	23.16	80.4	178.2	180	1, 3, 3, 3
CPU	Polar format, float32, corrected	17.8	-	-	-	-	-	-	-	-	-	-
CPU	Polar format, float32, gather	5.0	-	-	-	-	-	-	-	-	-	-
CPU	Polar format, float32, gather, corrected	11.6	-33.5	-46.7	0.9997	0.994	0.873	0.52	23.8	4.1	179	-42, -39, -36, -30

Table 13: Every configuration timed or measured, Iowa farmland: device time in seconds, with the pipelined seconds per image in parentheses where a loop was run; error and largest pixel difference (dB), 5 by 5 coherence statistics, amplitude and phase statistics over the brighter half of the pixels, and the error in four rings by distance from the scene center (quarters of the half-width, dB). A dash marks a configuration that was timed without saving its image.

Device	Configuration	Time	Error	Max	Coh mean	Coh p0.1	Coh min	Amp p99	Amp max	Ph p99	Ph max	Rings
L4	Exact BP, float16 profiles	63.8 (63.4)	-60.4	-63.3	1.0000	1.000	0.994	0.02	1.3	0.1	8	-60, -60, -61, -60
L4	Exact BP, float32 profiles	142.3 (141.8)	-60.5	-63.3	1.0000	1.000	0.994	0.02	1.2	0.1	8	-60, -60, -61, -60
L4	Factorized BP, float16 products	10.6	-57.2	-63.2	1.0000	1.000	0.974	0.03	1.4	0.2	9	-57, -57, -57, -57
L4	Factorized BP, float16 products, conv. filters	8.9 (9.6)	-57.2	-63.0	1.0000	1.000	0.974	0.03	1.4	0.2	9	-57, -57, -57, -57
L4	Factorized BP, float16 products, direct ramps	10.5	-	-	-	-	-	-	-	-	-	-
L4	Factorized BP, float32 levels, float16 final, CUDA kernels	2.0 (2.2)	-59.8	-62.9	1.0000	1.000	0.981	0.02	1.1	0.1	7	-60, -60, -60, -60
L4	Factorized BP, float32	22.0	-58.9	-63.0	1.0000	1.000	0.981	0.02	1.2	0.1	7	-59, -59, -59, -59
L4	Factorized BP, float32, conv. filters	10.3 (10.9)	-	-	-	-	-	-	-	-	-	-
L4	Factorized BP, float32, CUDA kernels	2.3 (2.5)	-60.0	-63.0	1.0000	1.000	0.981	0.02	1.1	0.1	7	-60, -60, -60, -60
L4	Factorized BP, TF32 products	14.5	-58.1	-63.2	1.0000	1.000	0.974	0.03	1.4	0.2	9	-58, -58, -58, -58
L4	Factorized BP, TF32 products, conv. filters	9.9 (10.5)	-	-	-	-	-	-	-	-	-	-
L4	Polar format, float16 product	1.2 (1.0)	3.0	6.1	0.4505	0.014	0.000	22.18	85.7	178.2	180	3, 3, 3, 3
L4	Polar format, float32	1.6 (1.4)	3.0	6.1	0.4505	0.014	0.000	22.18	85.1	178.2	180	3, 3, 3, 3
L4	Polar format, float32, corrected	2.1 (2.2)	-35.3	-35.3	0.9998	0.998	0.973	0.33	9.5	2.7	38	-40, -39, -37, -33
L4	Polar format, float32, gather	1.5 (1.2)	-	-	-	-	-	-	-	-	-	-
L4	Polar format, float32, gather, corrected	1.6 (1.7)	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass, bfloat16 storage	5.0	-46.9	-62.0	1.0000	1.000	0.957	0.10	1.8	0.6	30	-47, -47, -47, -47
TPU v5e	Factorized BP, six-pass products	10.5	-58.7	-63.0	1.0000	1.000	0.981	0.02	1.1	0.1	8	-59, -59, -59, -59
TPU v5e	Factorized BP, six-pass products, direct ramps	8.6	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass products	5.4	-46.9	-62.0	1.0000	1.000	0.957	0.10	1.8	0.6	30	-47, -47, -47, -47
TPU v5e	Factorized BP, single-pass products, conv. filters	10.9	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass products, direct ramps	4.4 (4.2)	-	-	-	-	-	-	-	-	-	-
TPU v5e	Factorized BP, single-pass products, fused kernels	2.2 (2.3)	-46.9	-61.9	1.0000	1.000	0.956	0.10	1.8	0.6	30	-47, -47, -47, -47
TPU v5e	Factorized BP, three-pass products	7.9	-58.7	-62.7	1.0000	1.000	0.981	0.02	1.1	0.1	8	-59, -58, -59, -59
TPU v5e	Factorized BP, three-pass products, direct ramps	6.1 (5.9)	-58.7	-63.0	1.0000	1.000	0.981	0.02	1.1	0.1	8	-59, -58, -59, -59
TPU v5e	Factorized BP, three-pass products, fused kernels	3.5 (3.7)	-58.7	-62.9	1.0000	1.000	0.981	0.02	1.1	0.1	8	-59, -59, -59, -59
TPU v5e	Polar format, single-pass product, gather	2.0 (3.0)	3.0	6.1	0.4505	0.014	0.000	22.18	85.1	178.2	180	3, 3, 3, 3
TPU v5e	Polar format, float32, gather	2.1 (2.0)	3.0	6.1	0.4505	0.014	0.000	22.18	85.1	178.2	180	3, 3, 3, 3
TPU v5e	Polar format, float32, gather, corrected	44.2	-35.3	-35.3	0.9998	0.998	0.973	0.33	9.5	2.7	38	-40, -39, -37, -33
TPU v5e	Factorized BP, single-pass, bfloat16 storage	3.2	-46.9	-62.0	1.0000	1.000	0.957	0.10	1.8	0.6	29	-47, -47, -47, -47
TPU v6e	Factorized BP, six-pass products	4.2	-58.9	-63.0	1.0000	1.000	0.981	0.02	1.2	0.1	8	-59, -59, -59, -59
TPU v6e	Factorized BP, six-pass products, direct ramps	4.2	-	-	-	-	-	-	-	-	-	-
TPU v6e	Factorized BP, six-pass products	2.9	-46.9	-62.0	1.0000	1.000	0.957	0.10	1.8	0.6	29	-47, -47, -47, -47
TPU v6e	Factorized BP, single-pass products, conv. filters	9.5	-	-	-	-	-	-	-	-	-	-
TPU v6e	Factorized BP, single-pass products, direct ramps	3.1 (2.9)	-	-	-	-	-	-	-	-	-	-
TPU v6e	Factorized BP, single-pass products, fused kernels	1.4 (1.5)	-46.9	-61.9	1.0000	1.000	0.956	0.10	1.8	0.6	30	-47, -47, -47, -47
TPU v6e	Factorized BP, three-pass products	3.5	-58.8	-62.7	1.0000	1.000	0.981	0.02	1.2	0.1	8	-59, -59, -59, -59
TPU v6e	Factorized BP, three-pass products, direct ramps	3.5 (3.3)	-58.8	-63.0	1.0000	1.000	0.981	0.02	1.2	0.1	8	-59, -59, -59, -59
TPU v6e	Factorized BP, three-pass products, fused kernels	1.9 (2.0)	-58.8	-62.9	1.0000	1.000	0.981	0.02	1.2	0.1	8	-59, -59, -59, -59
TPU v6e	Polar format, float32	0.9 (0.9)	3.0	6.1	0.4505	0.014	0.000	22.18	85.1	178.2	180	3, 3, 3, 3
TPU v6e	Polar format, float32, corrected	57.8 (59.3)	-35.3	-35.3	0.9998	0.998	0.973	0.33	9.5	2.7	38	-40, -39, -37, -33
TPU v6e	Polar format, single-pass product	0.8 (2.0)	3.0	6.1	0.4505	0.014	0.000	22.18	86.1	178.2	180	3, 3, 3, 3
TPU v6e	Polar format, float32, gather	0.9	-	-	-	-	-	-	-	-	-	-
TPU v6e	Polar format, float32, gather, corrected	57.8	-	-	-	-	-	-	-	-	-	-
CPU	Factorized BP, float32, conv. filters, direct ramps	191.0	-59.2	-62.9	1.0000	1.000	0.981	0.02	1.2	0.1	7	-59, -59, -59, -59
CPU	Factorized BP, float64, conv. filters, direct ramps	409.9	-60.1	-63.0	1.0000	1.000	0.981	0.02	1.1	0.1	7	-61, -60, -60, -60
CPU	Polar format, float32	9.9	3.0	6.1	0.4505	0.014	0.000	22.18	85.0	178.2	180	3, 3, 3, 3
CPU	Polar format, float32, corrected	12.1	-	-	-	-	-	-	-	-	-	-
CPU	Polar format, float32, gather	4.0	-	-	-	-	-	-	-	-	-	-
CPU	Polar format, float32, gather, corrected	8.2	-35.4	-35.4	0.9998	0.998	0.973	0.33	9.5	2.7	38	-40, -39, -37, -33

References

- [1] Umbra Lab, Inc., “Umbra Open Data Program,” 2023. [Online]. Available: <https://umbra.space/open-data>
- [2] R. K. Raney, “Synthetic aperture imaging radar and moving targets,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. AES-7, no. 3, pp. 499–505, 1971.
- [3] European Space Agency, “Copernicus Sentinel-2 Level-2A surface reflectance, tiles S2B_17PPK_20230723 and S2A_17PPK_20240223,” contains modified Copernicus Sentinel data 2023 and 2024, accessed October 2026 through the Earth Search STAC catalog. [Online]. Available: <https://registry.opendata.aws/sentinel-2-l2a-cogs/>
- [4] A. R. Fasih and T. D. R. Hartley, “GPU-accelerated synthetic aperture radar backprojection in CUDA,” in *Proc. IEEE Radar Conf.*, Washington, DC, 2010, pp. 1408–1413.
- [5] T. M. Benson, D. P. Campbell, and D. A. Cook, “Gigapixel spotlight synthetic aperture radar backprojection using clusters of GPUs and CUDA,” in *Proc. IEEE Radar Conf.*, 2012, pp. 853–858.
- [6] J. Park, P. T. P. Tang, M. Smelyanskiy, D. Kim, and T. Benson, “Efficient backprojection-based synthetic aperture radar computation with many-core processors,” in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC ’12)*, Salt Lake City, UT, 2012.
- [7] B. Cordes and M. Leeser, “Parallel backprojection: A case study in high-performance reconfigurable computing,” *EURASIP J. Embedded Syst.*, vol. 2009, art. 727965, 2009.
- [8] A. Capozzoli, C. Curcio, and A. Liseno, “Fast GPU-based interpolation for SAR backprojection,” *Progress In Electromagnetics Research*, vol. 133, pp. 259–283, 2013.
- [9] Y. Xu, Z. Zhang, L. Chen, and L. Yang, “The adaptive streaming SAR back-projection algorithm based on half-precision in GPU,” *Electronics*, vol. 11, no. 18, art. 2807, 2022.
- [10] O. Frey, C. Magnard, M. Rüegg, and E. Meier, “Focusing of airborne synthetic aperture radar data from highly nonlinear flight tracks,” *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 6, pp. 1844–1858, 2009.
- [11] O. Ponce, P. Prats, M. Rodriguez-Cassola, R. Scheiber, and A. Reigber, “Processing of circular SAR trajectories with fast factorized back-projection,” in *Proc. IEEE Int. Geosci. Remote Sens. Symp. (IGARSS)*, 2011, pp. 3692–3695.
- [12] A. W. Doerry, “Wavefront curvature limitations and compensation to polar format processing for synthetic aperture radar images,” Sandia National Laboratories, Albuquerque, NM, Rep. SAND2007-0046, 2007.
- [13] L. A. Gorham and B. D. Rigling, “Scene size limits for polar format algorithm,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. 52, no. 1, pp. 73–84, 2016.
- [14] C. V. Jakowatz, Jr. and N. Doren, “Comparison of polar formatting and back-projection algorithms for spotlight-mode SAR image formation,” in *Proc. SPIE 6237, Algorithms for Synthetic Aperture Radar Imagery XIII*, 2006, art. 62370H.

- [15] N. P. Jouppi *et al.*, “In-datacenter performance analysis of a tensor processing unit,” in *Proc. 44th Annu. Int. Symp. Computer Architecture (ISCA)*, Toronto, ON, 2017, pp. 1–12.
- [16] N. P. Jouppi *et al.*, “TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings,” in *Proc. 50th Annu. Int. Symp. Computer Architecture (ISCA)*, Orlando, FL, 2023.
- [17] T. Lu, Y.-F. Chen, B. Hechtman, T. Wang, and J. Anderson, “Large-scale discrete Fourier transform on TPUs,” arXiv:2002.03260, 2020.
- [18] T. Lu, T. Marin, Y. Zhuo, Y.-F. Chen, and C. Ma, “Nonuniform fast Fourier transform on TPUs,” in *Proc. IEEE 18th Int. Symp. Biomedical Imaging (ISBI)*, 2021.
- [19] A. G. M. Lewis *et al.*, “Large-scale distributed linear algebra with tensor processing units,” *Proc. Natl. Acad. Sci. USA*, vol. 119, no. 33, 2022.
- [20] L. A. Gorham and L. J. Moore, “SAR image formation toolbox for MATLAB,” in *Proc. SPIE 7699, Algorithms for Synthetic Aperture Radar Imagery XVII*, 2010, p. 769906.
- [21] J. W. McCorkle and M. Rofheart, “An order $N^2 \log(N)$ backprojector algorithm for focusing wide-angle wide-bandwidth arbitrary-motion synthetic aperture radar,” in *Proc. SPIE 2747, Radar Sensor Technology*, 1996, pp. 25–36.
- [22] S. Xiao, D. C. Munson, Jr., S. Basu, and Y. Bresler, “An $N^2 \log N$ back-projection algorithm for SAR image formation,” in *Conf. Rec. 34th Asilomar Conf. Signals, Systems and Computers*, 2000, pp. 3–7.
- [23] L. M. H. Ulander, H. Hellsten, and G. Stenström, “Synthetic-aperture radar processing using fast factorized back-projection,” *IEEE Trans. Aerosp. Electron. Syst.*, vol. 39, no. 3, pp. 760–776, 2003.
- [24] W. G. Carrara, R. S. Goodman, and R. M. Majewski, *Spotlight Synthetic Aperture Radar: Signal Processing Algorithms*. Norwood, MA: Artech House, 1995.
- [25] C. V. Jakowatz, Jr., D. E. Wahl, P. H. Eichel, D. C. Ghiglia, and P. A. Thompson, *Spotlight-Mode Synthetic Aperture Radar: A Signal Processing Approach*. Boston, MA: Kluwer, 1996.
- [26] L. R. Rabiner, R. W. Schafer, and C. M. Rader, “The chirp z-transform algorithm,” *IEEE Trans. Audio Electroacoust.*, vol. 17, no. 2, pp. 86–92, 1969.
- [27] T. T. Taylor, “Design of line-source antennas for narrow beamwidth and low side lobes,” *IRE Trans. Antennas Propag.*, vol. 3, no. 1, pp. 16–28, 1955.
- [28] J. F. Kaiser, “Nonrecursive digital filter design using the I_0 -sinh window function,” in *Proc. IEEE Int. Symp. Circuits and Systems*, 1974, pp. 20–23.
- [29] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: SIAM, 2002.
- [30] S. Wang and P. Kanwar, “BFloat16: The secret to high performance on Cloud TPUs,” Google Cloud Blog, Aug. 2019.
- [31] J. Bradbury *et al.*, “JAX: Composable transformations of Python+NumPy programs,” 2018. [Online]. Available: <https://github.com/google/jax>

- [32] S. K. Lam, A. Pitrou, and S. Seibert, “Numba: A LLVM-based Python JIT compiler,” in *Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC*, 2015.
- [33] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, “CuPy: A NumPy-compatible library for NVIDIA GPU calculations,” in *Proc. Workshop on Machine Learning Systems (LearningSys) at NIPS*, 2017.
- [34] A. H. Barnett, J. Magland, and L. af Klinteberg, “A parallel nonuniform fast Fourier transform library based on an ‘exponential of semicircle’ kernel,” *SIAM J. Sci. Comput.*, vol. 41, no. 5, pp. C479–C504, 2019.